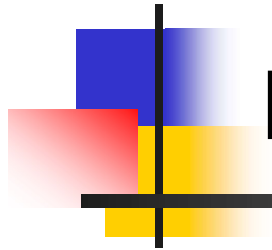
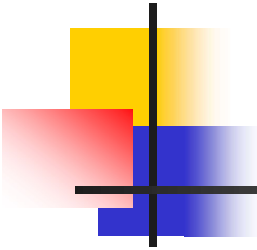


Fundamentos de Programação e Orientação à Objetos Utilizando C#



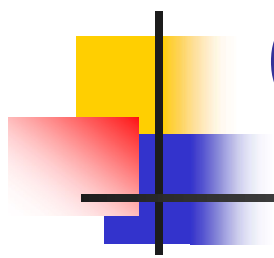
Professor: Geraldo Pereira de Souza

Obs: Esses slides foram produzidos para implementação em linguagem java e estão sendo atualizados para C#. Caso verifique algum erro ou em caso de dúvidas e/ou sugestões favor informar ao professor.



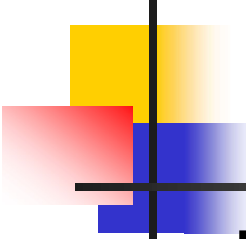
O que é C#?

- Uma linguagem de programação
 - ⇒ É uma linguagem de programação orientada a objetos, baseada nas linguagens C++ e com a agilidade do Visual Basic;
 - ⇒ Linguagem fortemente tipada.



Características

- Fortemente tipada;
- Linguagem segura;
- Linguagem que será compilada para rodar na CLR do .Net.



Características

- Eliminar práticas de codificação que afetam a robustez do código:
 - ⇒ Aritmética de ponteiros;
 - ⇒ Alocação e liberação de memória;
- Linguagem com facilidades para programação em rede;
- Codificação rápida com a utilização de IDEs como *Visual Studio 2022*, *Vs Code*, etc;

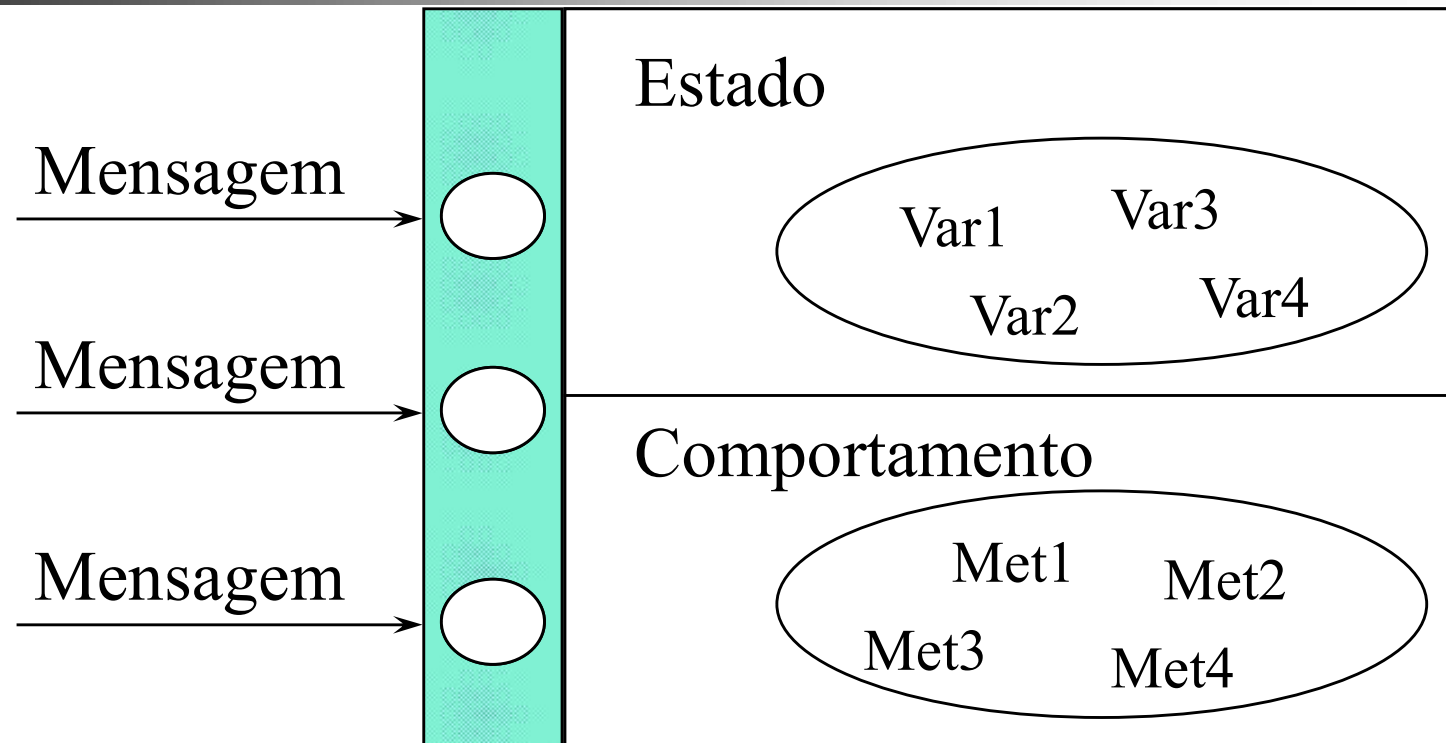


Características:

- Simplicidade

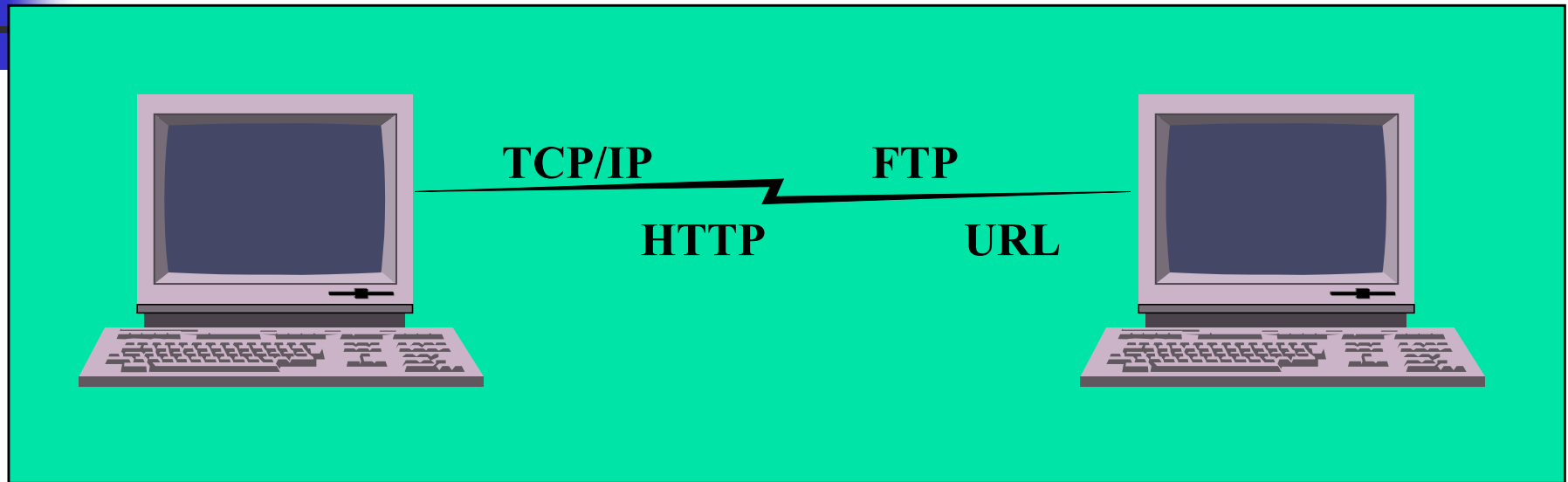
- ⇒ Sintaxe/linguagem do tipo c-like e parecida com C/C++, mas:
 - Não possui herança múltipla;
 - **Faz coleta automática de lixo em memória (“Garbage collector”);**
- ⇒ Agilidade de desenvolvimento do VB;
- ⇒ Linguagem “enxuta”
 - interpretador e biblioteca padrão pequenos;

Tecnologia de Objetos

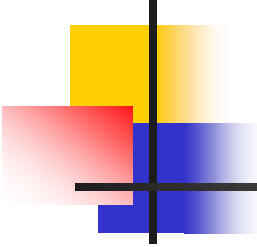


- Interfaces bem definidas
- Reusabilidade

Suporte a Redes



- Bibliotecas padronizadas com suporte para protocolos
- Facilidade de interconexão
- Fácil acesso a recursos remotos



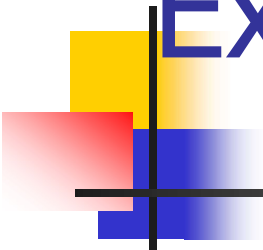
Linguagem Robusta e Segura

- Tipagem estática e forte
 - ⇒ Verificação em tempo de compilação
- Facilidade programação distribuída e escalável
- Várias bibliotecas e API's desenvolvidas
 - ⇒ Na internet existe **"muita coisa pronta"**

Gerenciamento de memória na CLR

■ O coletor de lixo ("Garbage Collector")

- ⇒ É um processo que gerencia a alocação e liberação de memória;
- ⇒ Durante cada ciclo de CPU ("Idle time") da CLR (***Common Language Runtime***), o processo verifica se há na memória elementos que não estão sendo referenciados, caso haja estes são liberados.



Exemplo 1

```
Using System;
```

```
// Comentário de uma linha
```

```
/*
```

```
    Comentário em  
    múltiplas linhas
```

```
*/
```

```
public class Hello
```

```
{ /** Comentário */
```

```
    public static void Main (string args[])
```

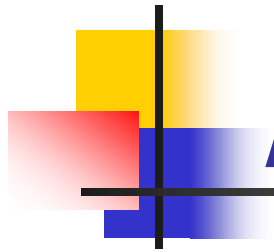
```
{
```

```
    Console.WriteLine("Alô classe!");
```

```
    Console.ReadKey();
```

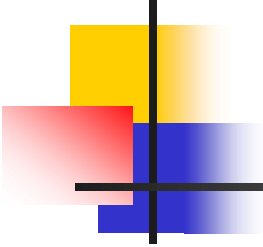
```
}
```

```
}
```



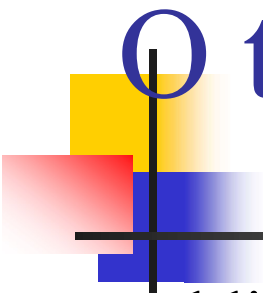
// Comentário de uma linha

- ⇒ Comentários seguem a mesma sintaxe do C++:
 - ⇒ `//` Uma única linha
 - ⇒ `/* */` Múltiplas linhas



```
public class Hello {
```

- **class** é a palavra reservada que marca o início da declaração de uma classe.
- **public** é um modificador, indica que a classe pode ser instanciada e acessada por outras classes.
- **Hello** é o nome da classe



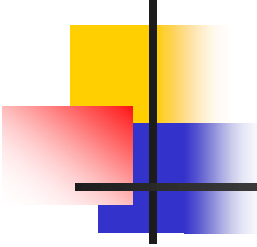
O trecho do programa

```
public static void Main (String [] args){  
    Console.WriteLine("Alô classe!");  
}
```

O método Main é o método principal da aplicação. Ele é o primeiro método a ser executado.



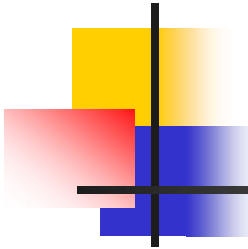
- Indica que o método pode ser executado diretamente sem a criação de um objeto.
- Será abordado mais adiante.



void

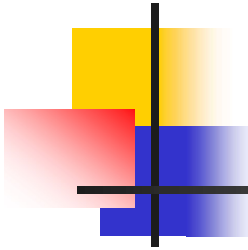
- Semelhante ao `void` C++ ou C, é o valor de retorno da função, quando a função não retorna nenhum valor.
- Se o método retorna um valor que não é `void`, `int` por exemplo, um `return` dentro do método é necessário.

```
public double quadrado(int x){  
    return x*x;  
}
```



Main

- É o método principal da aplicação e é onde a execução se inicia.
- Declarações possíveis do Main:
 - static int Main()*
 - static void Main()*
 - static void Main(string[] argumento)*
 - static int Main(string[] argumento)*



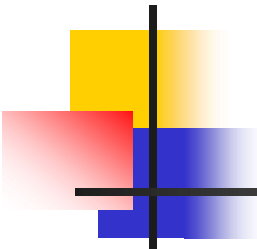
Main

- Outras declarações possíveis:

static void Main()

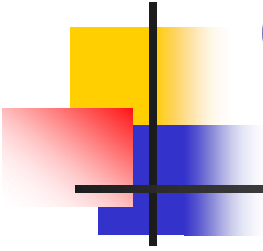
private static void Main()

public static void Main()



(string args[])

- Argumentos que são passados para o programa principal pela linha de comando.
- Ele é um vetor de **strings**. O primeiro argumento ocupa a posição 0, exemplo:
 - ⇒ Arg primeiro aula de OO



```
Console.WriteLine("Hello Internet!");
```

- Imprime um objeto do tipo string na saída padrão e posiciona uma linha abaixo.
- Para imprimir o conteúdo de um atributo x:

```
Console.WriteLine("Valor de x = {0}", x);
```

- Mais de um atributo:

```
Console.WriteLine("Valores de x = {0} e y = {1}", x, y);
```



Declaração de Variáveis

*<Modificador> <Tipo> <nome> = * <valor inicial>*

- *Opcional para classes e obrigatório em métodos*

• *Ex: public int numeroCandidatos = 1000;*

ou

public int numeroCandidatos;

numeroCandidatos = 1000;

- Variáveis podem ser declaradas em qualquer posição do código fonte.



Declaração de Variáveis

*<Modificador> <Tipo> <nome> = * <valor inicial>*

- *Opcional para classes e obrigatório em métodos*

• *Ex: public double nota = 5.6;*

ou

public double nota ;

nota = 5.6;

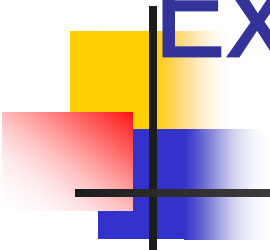
public double x=10, y=5, z=7;



Estrutura mínima de um programa em CSharp

```
class <nome> {  
    public static void Main (<parametro>) {  
        <declarações>  
        <comandos>  
    }  
}
```

Onde Main: método por onde se inicia a execução
public: qualificador de acesso
static: indica que Main se aplica à classe
void: indica que Main não retorna um valor



Exemplo 2

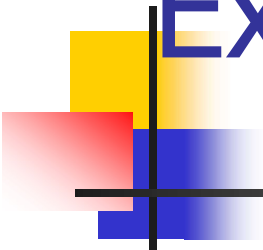
```
class LeImprime {  
    /** Lê e imprime um string */  
    public static void Main (String[] args) {  
        string nome;  
        nome = Console.ReadLine();  
        Console.WriteLine (nome)  
    }  
}
```

Exemplo 3

```
class LeImprimeIdade {  
    /** Lê e imprime um string */  
    public static void Main (String[] args) {  
        string entrada;  
        Console.Write("Digite sua idade: ");  
        entrada = Console.ReadLine();  
        int idade = int.Parse(entrada);  
        Console.WriteLine ("Idade = {0}", idade);  
        Console.ReadLine();  
    }  
}
```


Exemplo 4

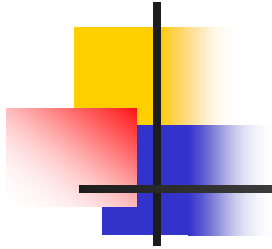
```
class LeImprimeNota {  
    /** Lê e imprime um string */  
    public static void Main (String[] args) {  
        string entrada;  
        Console.Write("Digite a sua nota:");  
        entrada = Console.ReadLine();  
        double nota = Double.Parse(entrada);  
        Console.WriteLine ("Nota = {0}", nota);  
        Console.ReadKey ();  
    }  
}
```



Exemplo 5

```
class LeImprimeNota {  
    /** Lê e imprime um string */  
    public static void Main (String[] args) {  
        int x = int.Parse(Console.WriteLine("Digite o valor de x:"));  
        int y = int.Parse(Console.WriteLine("Digite o valor de y:"));  
        double z = x/y;  
        Console.WriteLine ("Z = {0}", z);    Console.ReadKey ();  
    }  
}
```

Laboratório



Ex 01: Instruções em `mod01\lab\Exercicio01.html`

Ex 02: Instruções em `mod01\lab\Exercicio02.html`

IDE

The screenshot shows the Visual Studio Installer application window. The title bar reads "Visual Studio Installer". Below the title bar, there are tabs for "Instalado" (selected) and "Disponível".

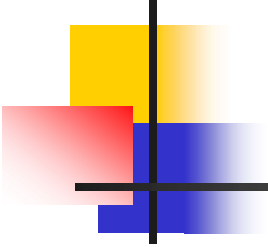
The main content area displays two items:

- Visual Studio Community 2022**: Shows the download and verification progress (1.5 GB of 1.98 GB, 74% complete) and the installation progress (package 367 of 438, 74% complete). A "Pausar" button is visible. Below the progress bars, it says "Microsoft.VisualStudio.Windows.Forms" and "Iniciar após a instalação" (checked). A link "Notas sobre a versão" is also present.
- Ferramentas de Build do Visual Studio 2019**: Shows version 16.11.29 and indicates that an update is available (16.11.33). Buttons for "Modificar", "Iniciar", "Mais", and "Atualizar" are visible.

On the right side, there is a section titled "Notícias do Desenvolvedor" (Developer News) with several articles and a link to "Ver mais notícias do desenvolvedor Microsoft...". At the bottom right, there is a section for help and support, including a link to the "Microsoft Developer Community" and a link to "Suporte do Visual Studio".

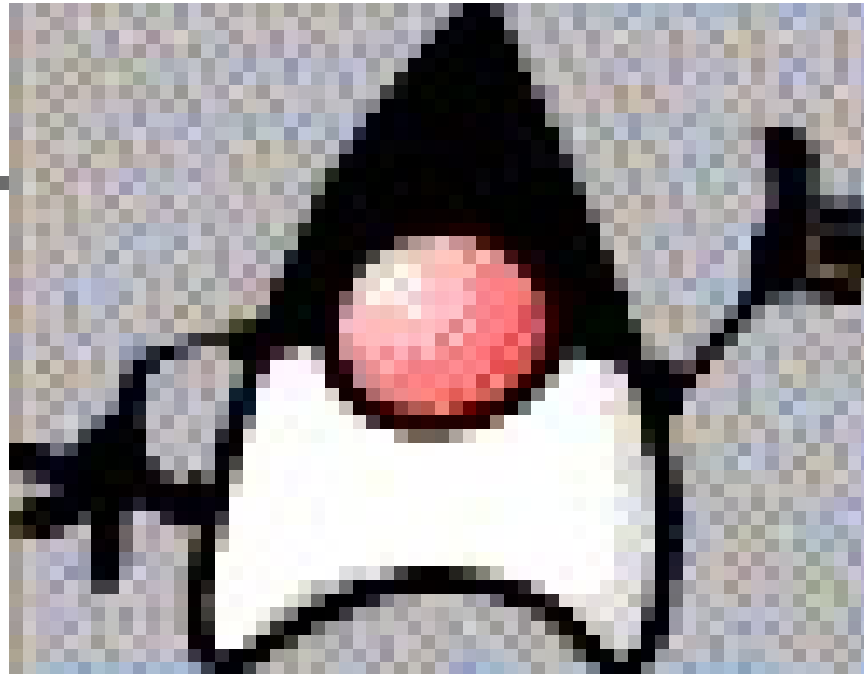
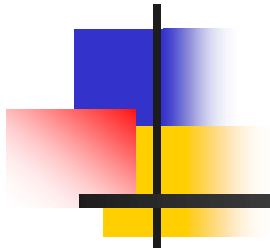
The Windows taskbar at the bottom shows the Start button, a search bar, and several application icons (File Explorer, Chrome, Edge, Visual Studio). The system tray on the right shows the time (20:14), date (quarta-feira 07/02/2024), and various system icons.

Antes de prosseguir, assegure-se que sabe:

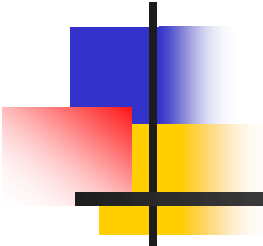


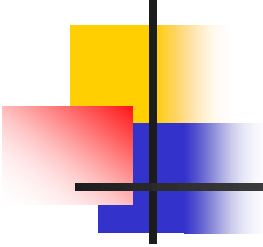
- Definir as principais características da linguagem.
- Compilar e rodar aplicações.
- Identificar os principais pontos de uma aplicação.
- Definir porque a padronização é importante no desenvolvimento de aplicações

Parte 2: Estrutura da Linguagem



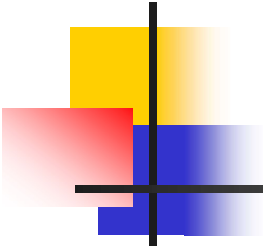
Estrutura da linguagem

- 
- ✓ *Convenção identificadores*
 - ✓ *Tipos primitivos*
 - ✓ *Operadores*
-



Identificadores

- Obs: Não pode ser palavra reservada.
- O primeiro caracter pode ser uma uma letra: A-Z ou a-z ou sublinhado (_)
- E o restante podem ser os mesmos do grupo anterior ou caracteres numéricos (0-9).



Identificadores

■ Exemplos:

Identificadores válidos:

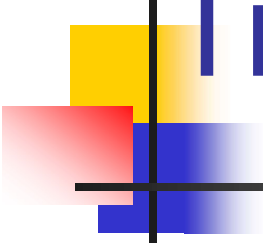
userName	User_name	
userName2	_sys_var1	\$change

Identificadores inválidos:

user/Name	-User_name	
5userName	_sys*var1	çchange

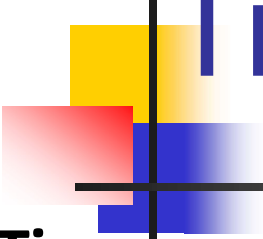
Obs: c# é case sensitive:

nomeUsuario ≠ NomeUsuario



Tipos básicos/primitivos

Tipo	Descrição	Faixa	
bool	Booleano	<i>true</i> ou <i>false</i>	
char	Unicode de 16 bits	U+0000 a U+ffff	
sbyte	Inteiro de 8 bits	-127 a +128	
short	Inteiro de 16 bits	-2^{15} a $+2^{15} - 1$	
int	Inteiro de 32 bits	-2^{31} a $+2^{31} - 1$	
long	Inteiro de 64 bits	-2^{63} a $+2^{63} - 1$	
float	32-bit	IEEE754	IEEE754
double	64-bit	IEEE754	IEEE754



Tipos básicos/primitivos

Tipo	Descrição	Faixa
string	<i>string</i> de caracteres <i>unicode</i>	
byte	Inteiro de 8 bits sem sinal	0 a $+2^8 - 1$
ushort	Inteiro de 16 bits sem sinal	0 a $+2^{16} - 1$
uint	Inteiro de 32 bits sem sinal	0 a $+2^{32} - 1$
ulong	Inteiro de 64 bits sem sinal	0 a $+2^{64} - 1$
decimal	Inteiro de 96 bits	
half	Representa um ponto flutuante de 16 bits. Novo tipo introduzido no C#.	
object	Classe base	

Obs: nome de variável não pode ser uma palavra reservada da linguagem: *if*, *for*, *while*, etc. Consultar livro/referência da ementa.

Como declarar e atribuir valores a atributos

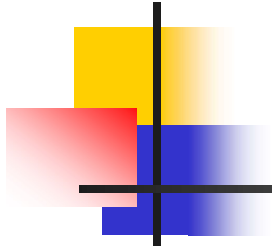
```
public class Assign {  
    static void Main (String args[]) {  
        int x, y;                // declarando variáveis inteiras  
        float z = 3.414f;        // declarando e atribuindo ponto flutuante  
        double w = 3.1415;       // declarando e atribuindo double  
        bool truth = true;       // declarando e atribuindo booleano  
  
        char c;                  // declarando variável caracter  
        string str;              // declarando variável string  
        c = 'A';                 // atribuindo um valor char a variável  
        str = "Oi pessoal!";     // atribuindo uma cadeia de caracteres  
        x = 6;  
        y=1000;                  // atribuindo valores a variável  
    }  
}
```



Tópicos adicionais

- Documentação C#
- Listas de discussão
 - Listas de discussão

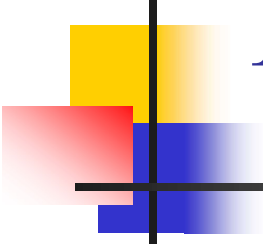
Laboratório



Instruções em `mod01\lab\Exercicio03.html`

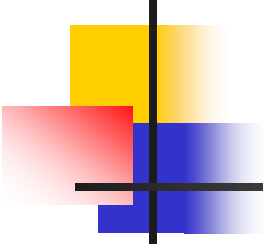
Classes e Objetos





Tipo Abstrato de Dados (TAD)

- ⇒ Tipos simples de dados são grupos de valores indivisíveis (como os tipos básicos/primitivos de uma linguagem de programação OO);
- ⇒ TAD segundo Ziviani: um modelo matemático, acompanhado de operações definidas sobre o modelo;
- ⇒ De modo simplificado, uma TAD pode ser definida como um conjunto de dados e operações que atuam sobre esses dados.

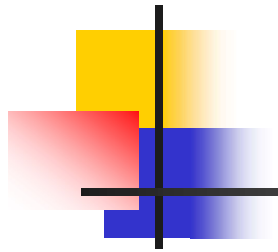


Classes e Objetos

Classe: é um protótipo (modelo) usado para criar vários objetos semelhantes.

Objetos: São instâncias de uma classe e representam entidades do mundo real. Os objetos são criados a partir de uma classe.

O que uma classe tem a ver como um TAD?



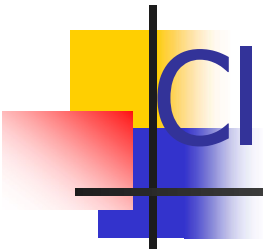
Classes e Objetos

Classe: é um protótipo (modelo) usado para criar vários objetos semelhantes.

Objetos: São instâncias de uma classe e representam entidades do mundo real. Os objetos são criados a partir de uma classe.

O que uma classe tem a ver como um TAD?

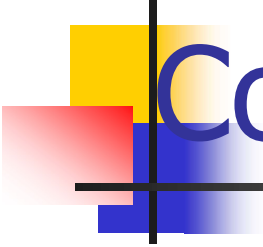
Uma classe é uma TAD “chique” em OO. Chique porque a classe define os atributos e as operações que podem operar sobre esses atributos. Além disso define métodos públicos (métodos de instância) que podem ser acessados fora da classe.



Classe

- Permite a instanciação de um ou mais objetos de um mesmo tipo.

Veículo
-chassi : long #ano : int +modelo : int
+ligar() +acelerar() +passarMarcha()



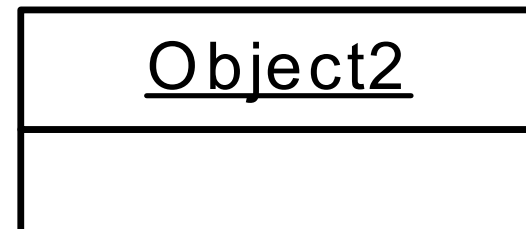
Convenções da UML

Veículo
-chassi : long #ano : int +modelo : int
+ligar() +acelerar() +passarMarcha()

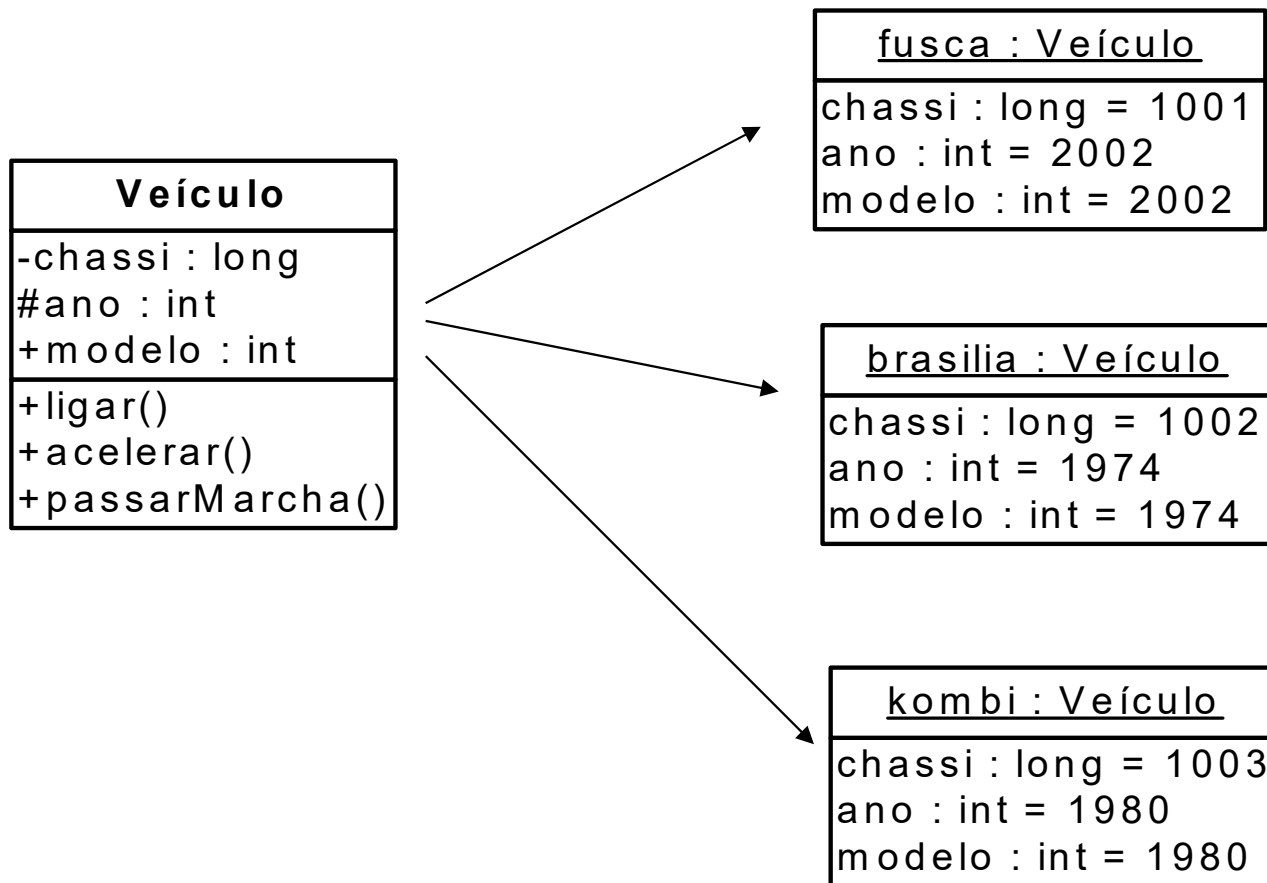
- - acesso privado
- # acesso protegido
- + acesso público

Objeto

- ⇒ Possui um identificador único
- ⇒ Esconde os dados (variável de instância) e métodos privados
- ⇒ Executa operações (métodos) quando solicitado
- ⇒ Pode ter uma interface pública



Classes e Objetos





Abstração e Encapsulamento

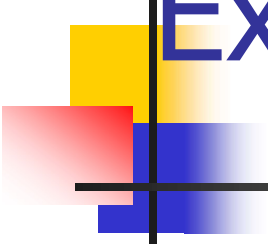
Abstração de dados :

Não é importante saber como os dados estão armazenados. Utiliza-se operações para acessar ou executar alguma operação do objeto.

Ex: Em um Cash o usuário saca dinheiro, ele não precisa saber as operações envolvidas no processo.

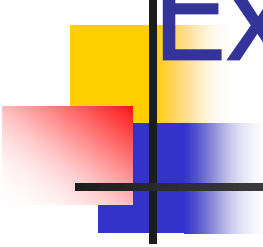
Encapsulamento:

Somente atributos e métodos públicos podem ser manipulados externamente.



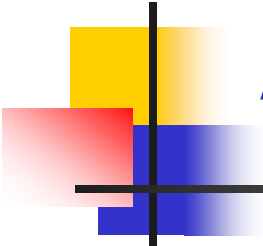
Exemplos de objetos

- Carro
- Bicicleta
- Computador
- Conta de cliente
- Bola
- *Outros exemplos*
 - *Uma transação de pix seria um objeto?*
 - *A conta que você possui em algum banco seria um exemplo de objeto?*



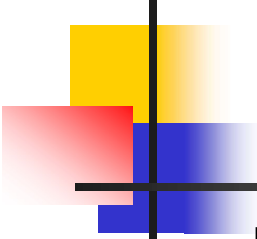
Exemplos de objetos

- *Outros exemplos*
 - *Você já viu uma conta bancária? Aliás o que é uma conta bancária?*
- *Um objeto do tipo cartão bancário e conta bancária são do mesmo tipo?*
- *De um ponto de vista mais prático sobre a ótica da programação, será que existe mesmo classes no mundo real ou existem somente objetos???*



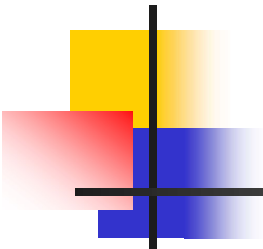
Atributos e métodos

- **Atributos** definem as características de um objeto. Ex: Classe Conta: numero, titular, saldo, etc;
- **Métodos:** realizam tarefas, cálculos, etc.:
 - calcularCpmf(), emitirExtrato()
- Obs: Por enquanto, sintaticamente o que diferencia um atributo de um método são os parênteses.
- Por que não chamar métodos de funções? Porque a partir de agora nós somos chiques. Vamos programar usando OO. O termo função remete à programação estruturada que é orientada à funções e procedimentos.



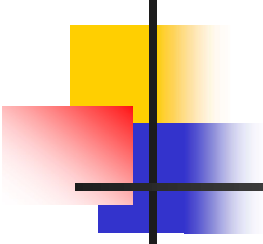
Programa Orientado a Objetos

- Um programa é construído a partir de uma *coleção de objetos*
- Classes são abstrações semelhantes a “fábricas” a partir das quais construímos *objetos*, chamados *instâncias da classe*
- Classes contém *membros* que são:
 - ⇒ *Atributos*: definem o *estado dos objetos*
 - ⇒ *Métodos*: executam operações e operam sobre os dados/atributos mudando o estado dos objetos



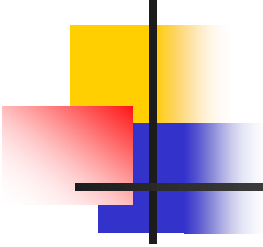
Codificação de uma classe em C#

```
public class Conta {  
    public int numero;  
    public string titular;  
    public double saldo;  
    public void imprimeSaldo () {  
        Console.WriteLine("Saldo: "+saldo);  
    }  
    public void imprimeTitular () {  
        Console.WriteLine("Titular: "+titular);  
    }  
}
```



Como criar objetos?

```
public class TestaConta {  
    public static void Main(String[] a) {  
        Conta c1  = new Conta();  
        c1.numero = 1;  
        c1.titular = "José";  
        c1.saldo = 1000;  
        c1.imprimeSaldo();  
        c1.imprimeTitular();  
        Conta c2=new Conta();  
        c2.numero=2;  
    }  
}
```



Método Construtor

É o método invocado automaticamente no momento da criação de um objeto.

Exemplo de criação de objetos:

```
Conta c1 = new Conta();
```

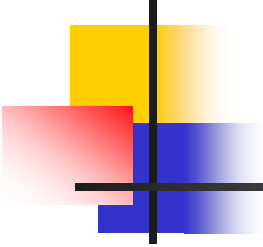
```
c1.numero = 1;
```

```
c1.titular = "FHC"
```

```
c1.saldo = 10000.00
```

ou usando o método construtor

```
Conta c1 = new Conta(1, "FHC", 10000.00)
```



Método Construtor

Dentro da classe:

```
public class Conta{
```

```
    private int numero;
```

```
    private String titular;
```

```
    private double saldo;
```

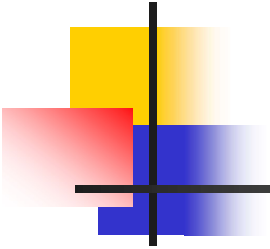
```
    public Conta(int n, String t, double saldo){
```

```
        numero = n; titular = t; this.saldo = saldo;
```

```
    }
```

```
    ..... Continua a definição da classe ....
```

```
}
```



Método gets e sets

```
public class Conta{
```

```
..... Definição dos atributos e construtores ....
```

```
public int getNumero(){
```

```
    return numero;
```

```
}
```

```
public void setNumero(int numero){
```

```
    this.numero=numero;
```

```
}
```

```
public double getSaldo(){
```

```
    return saldo;
```

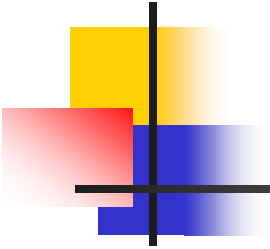
```
}
```

```
public void setSaldo(double saldo){
```

```
    this.numero=numero;
```

```
}
```

```
..... continua....
```

Método gets e sets

public class Conta{

..... Definição dos atributos e construtores

public String getTitular(){

return saldo;

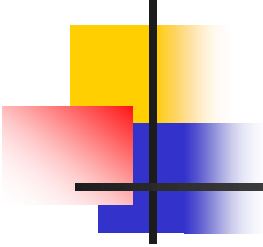
}

public void setTitular(String titular){

this.titular=titular;

}

}



Método Construtor e *this()*

```
public class Ponto{
```

```
    private string descricao; private int x; private int y;
```

```
    public Ponto(){
```

```
        this.descricao = "p"; this.x = 5; this.y = 5;
```

```
    }
```

```
    public Ponto(string descricao): this(){
```

```
        this.descricao = descricao;
```

```
    }
```

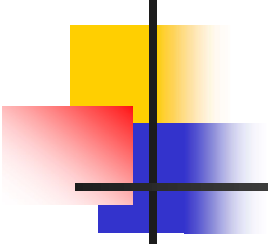
```
    public Ponto(string descricao, int x, int y): this(descricao){
```

```
        this.x = x;
```

```
        this.y = y;
```

```
    }
```

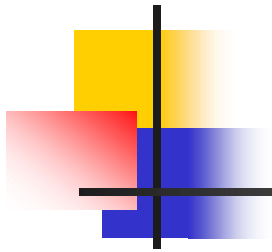
..... continua.....



Método Construtor

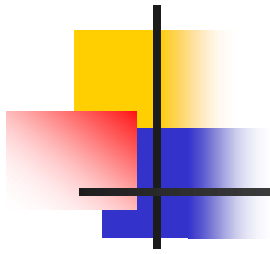
É o método que é executado na classe no momento da criação de um objeto.

- Qual a finalidade dos métodos construtores?
 - Inicializar os valores dos atributos
 - Executar um ou mais métodos na criação de um objeto



Método Construtor

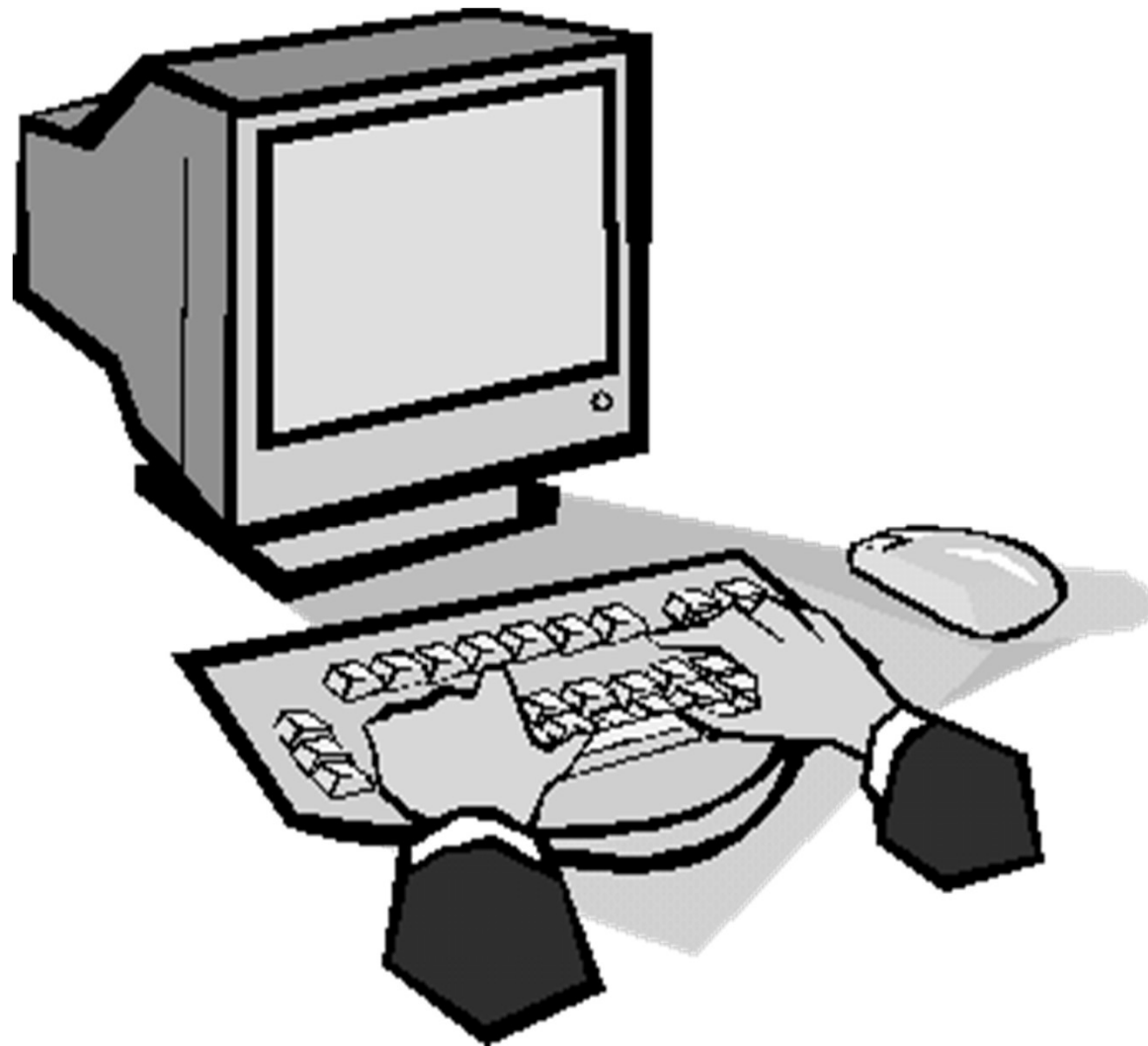
- Sintaxe do construtor:
 - Método sem retorno que recebe o mesmo nome da classe
 - Valores de parâmetros são passados no momento da criação
 - Uma classe pode ter um número ilimitado de métodos construtores. A execução do construtor vai se dar de acordo com o número e tipo dos parâmetros (que definem **a assinatura** do método) passados no momento da criação do objeto.



Método Construtor

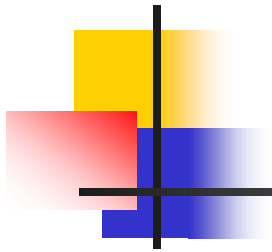
- Toda classe tem um construtor default, sem parâmetros, se nenhum é especificado;
- Se for declarado pelo menos um método constructor na classe, o constructor sem parâmetros deve ser declarado explicitamente;
- Não há limites para o número de construtores em uma classe;
- A palavra `this` pode ser utilizada para invocar outro construtor da mesma classe;
- A palavra **base** faz referência a um método constructor de uma superclasse;

Revisão e Laboratório



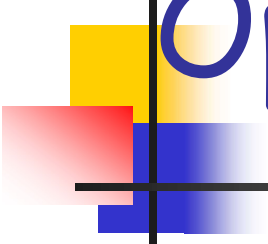
Módulo 03 - Continuação sobre Fundamentos da Linguagem





Operadores em C#

Unários	++ -- + - ! ~ ()
Aritméticos	* / % + -
Comparação	< <= > >=
Short-circuit	&&
Ternário	?:
Atribuição	= “op=”



Operadores Aritméticos

Operador	Significado	Exemplo
+	Adição	7 + 2
-	Subtração	3 - 2
*	Multiplicação	8 * 2
/	Divisão	6 / 3
%	Resto da divisão	7 % 2 => 1



Operadores Aritméticos

Oper	Uso	Descrição
++	op++	Incrementa op de 1 unidade
--	op--	Decrementa op de 1 unidade

Qual a diferença entre op++ e ++op?

```
int i = 5;
```

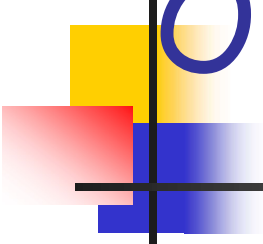
```
int x = i++; // x vale 5
```

ou

```
int i = 5;
```

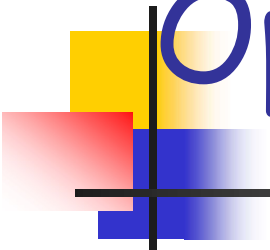
```
int x = ++i; // x vale 6
```

Obs: A diferença se dá devido a ordem de avaliação da expressão.



Operadores Relacionais

Operador	Significado	Exemplo
>	Maior que	<code>x > 3</code>
>=	Maior ou igual	<code>x >= 3</code>
<	Menor que	<code>x < 4</code>
<=	Menor ou igual	<code>x <= 7</code>
==	Igual a	<code>x == 8</code>
!=	Diferente	<code>x != 7</code>



Operadores Lógicos

Oper Uso

&& op1 && op2

|| op1 || op2

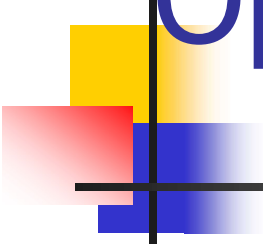
! !op

Retorna true se

op1 **e** op2 forem true. Só avalia op2, **se** op1 for true.

op1 **ou** op2 for true (ou ambos). Só avalia op2, **se** op1 for false.

op for false.



Operador ternário

Condição?Comando1:Comando2

`(x>y)?Console.WriteLine(x):Console.WriteLine(y)`

equivalente a

```
if (x > y){  
    Console.WriteLine(x);  
}  
else{  
    Console.WriteLine(y);  
}
```

Array



Estrutura de dados que pode conter um conjunto homogêneos de elementos.

Características:

- Tamanho fixo.
- O índice do primeiro elemento: 0

Definição:

```
int[] lista = {10, -2, 3};
```

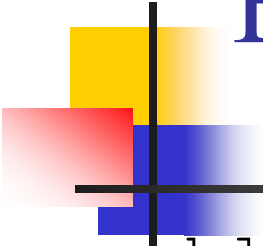
ou

```
int lista[] = new int[3];
```

```
lista[0] = 10; lista[1] = -2;
```

```
lista[1] = 3;
```

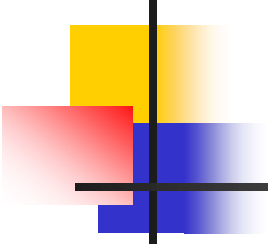
Propriedade Length: retorna o número de elementos



Exemplo de uso - array

```
public class TesteArray
{
    public static void main(string[] args)
    {
        int[] a = { 1, 12, -3, 4, 15 };

        for(int i=0; i< a.Length; i++)
            Console.WriteLine("a[{0}]:", a[i]);
    }
}
```



Array multidimensionais

```
int[][] coords = new int[5][4];
```

Para acessar:

```
coords[0][0] = 1;
```

```
coords[0][1] = 2;
```

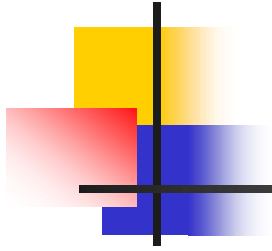
```
.....
```

```
....
```

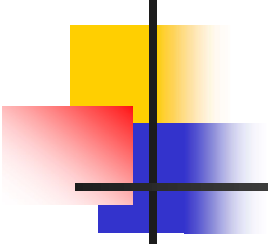

Inicialização de elementos no array

Tipo	Valor inicial	Tipo	Valor Inicial
byte	0	short	0
int	0	long	0L
float	0.0f	double	0.0d
char	'\u0000'	boolean	false
referência a objeto	null		

Laboratório



Orientações em `mod03\lab\Lab01.html`



Módulo 04 (Opcional) - Controle de Fluxo

Categoria

decisão

loop

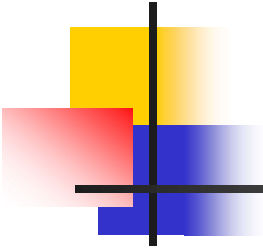
diversos

Comando

if-else, switch-case

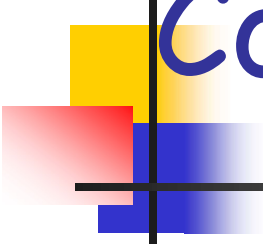
for, while, do-while

break, continue, label: , return



Controle de Fluxo - if

```
if  (expressão booleana) {  
    comando ou bloco  
}  
else {  
    comando ou bloco  
}
```



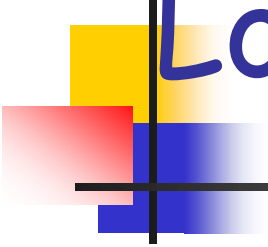
Controle de Fluxo - switch

```
switch (variavel) {  
    case expressão1:  
        comando ou bloco  
        break;  
  
    . . .  
  
    case expressãon:  
        comando ou bloco  
        break;  
  
    default:  
        comando ou bloco  
        break;  
}
```



Exemplo de uso - switch

```
public class Switch
{
    public static void main (String args[])
    {
        if (args.Length < 1)
        {
            Console.WriteLine("Informe um numero entre 1 e 3");
            Console.exit(0);
        }
        // conversao de String para inteiro
        int a = Integer.parseInt(args[0]);
        switch (a)
        {
            case 1: Console.WriteLine ("Numero UM");
                    break;
            case 2: Console.WriteLine ("Numero DOIS");
                    break;
            case 3: Console.WriteLine ("Numero TRES");
                    break;
            default: Console.WriteLine ("NUMERO INVALIDO");
        }
    }
}
```



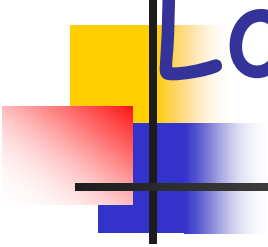
Loops - while

```
while (expressão booleana) {  
    comando ou bloco  
}
```



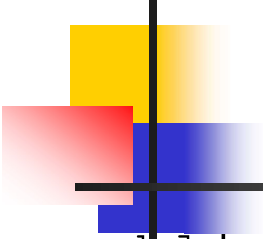
Exemplo de uso - while

```
public class ListaParametros
{
    public static void main (String args[])
    {
        int totalParametros = args.length;
        Console.WriteLine("Total : "+totalParametros);
        int i = 0;
        while (i < totalParametros)
        {
            Console.WriteLine("Parametro "+i+": "+args[i]);
            i = i + 1;
        }
    }
}
```

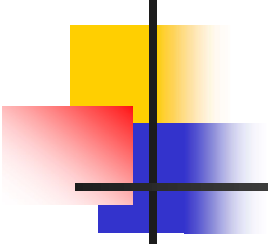
Loops - do while

```
do {  
    comando ou bloco  
    expressão de iteração;  
} while (expressão booleana);
```



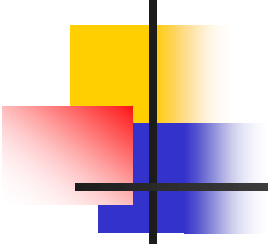
Exemplo de uso - do while

```
public class ListaParametros
{
    public static void main (String args[])
    {
        int totalParametros = args.length;
        Console.WriteLine("Total : "+totalParametros);
        int i = 0;
        if (totalParametros > 0)
        {
            do
            {
                Console.WriteLine("Par. "+i+": "+args[i]);
                i = i + 1;
            }
            while (i < totalParametros)
        }
    }
}
```



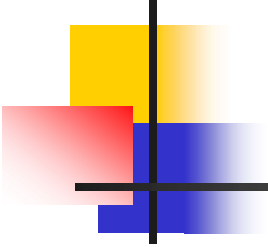
Loops - for

```
for(int_statament; boolean_cond; step)
{
    << comandos >>
}
```



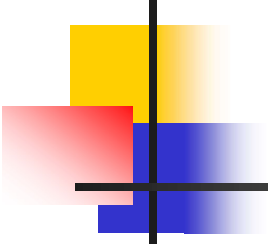
Loops - for com Break

```
for(int_statament; boolean_cond; step)
{
    << comandos >>
    break;
}
```



Loops - for com Continue

```
for(int_statament; boolean_cond; step)
{
    << comandos >>
    continue;
}
```

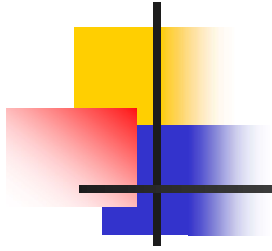


Loops - for com label

```
label:for(int_statament; boolean_cond;  
step)  
{  
    << comandos >>  
    continue label;  
}
```

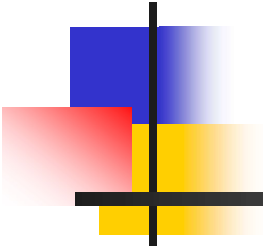
Obs: Exercícios sobre loops no diretório loop.

Laboratório

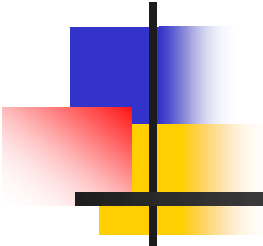


Orientações em `mod04\lab\Exercicio01.html`

Interfaces, funções/métodos estáticos em C#

- 
- Conceito de interface;
 - Declaração de interface em C#;
 - Métodos abstratos;
 - Passagem de parâmetro por valor;
 - Passagem de parâmetro por referência;

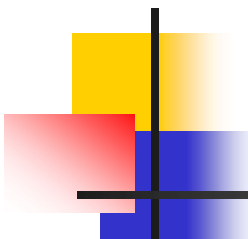
Funções/métodos estáticos em C#

- 
- Exemplo de função em C#
 - Regra para criar uma função
 - Passagem de parâmetro
 - Passagem de parâmetro por valor
 - Passagem de parâmetro por referência



Exemplo 1:

```
public class Programa{  
    public static void saudacaolnicial(){  
        Console.WriteLine("\nSeja bem vindo usuário! ");  
    }  
    public static void saudacaoFinal(){  
        Console.WriteLine ("\nVolte sempre usuário! ");  
    }  
    public static void Main(){  
        saudacaolnicial();  
        Programa.saudacaoFinal();  
        Console.WriteLine("\nPressione qualquer tecla para continuar... ");  
        Console.ReadKey();  
  
    }
```



Exemplo 2: com parâmetros

```
public class Programa
```

```
public static int somaNumeros(int x, int y){
```

```
int resultado = x+y;
```

```
return resultado;
```

```
}
```

```
public static void Main (){
```

```
int numero1, numero2;
```

```
Console.WriteLine("\nDigite o primeiro número: ");
```

```
numero1 = int.Parse(Console.ReadLine());
```

```
Console.WriteLine("\nDigite o segundo número: ");
```

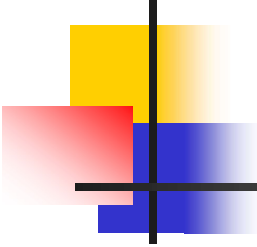
```
numero2 = int.Parse(Console.ReadLine());
```

```
Console.WriteLine("\nSoma dos números: {0} ", somaNumeros(numero1, numero2));
```

```
Console.ReadKey();
```

```
}
```

```
}
```



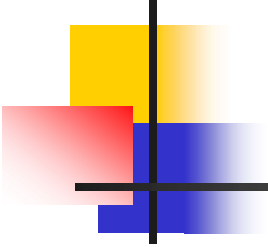
Forma Geral para Função parâmetros (procedimento)

```
public static void nomeDaFuncao(tipo nomeParametro1, tipo,  
    nomeParametro2, ...){
```

```
// Corpo da função
```

```
// Não pode colocar return porque a função declara retorno do tipo void
```

```
}
```



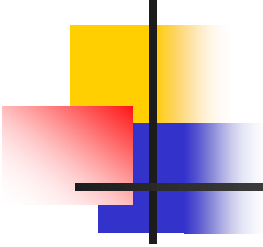
Forma Geral para Função

```
public static tipo_retorno nomeDaFuncao(tipo nomeParametro1, tipo,  
    nomeParametro2, ...){
```

```
    // Corpo da função
```

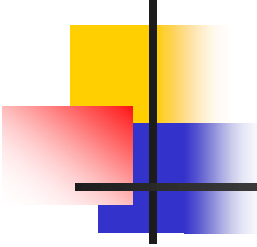
```
    return valor;    // valor tem que ser do mesmo tipo de tipo_retorno
```

```
}
```



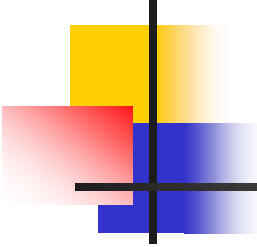
Por que usar função?

- Reuso de código: função pode ser chamada várias vezes dentro da própria classe ou a partir de outras classes;
- Facilita manutenção;
- Facilita a legibilidade do programa. Mais fácil de ser entendido;



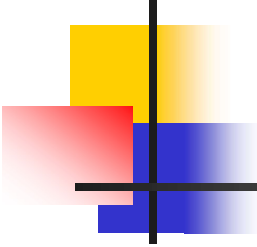
Funções Escopo e Passagem de Parâmetros

- Escopo de Variáveis
- Passagem de parâmetro por valor
- Passagem de parâmetro por referência



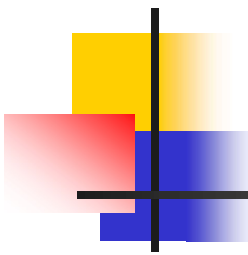
Funções Escopo e Passagem de Parâmetros

- > Escopo é a área de atuação de uma determinada variável.
- > Em C# o escopo de uma variável é definido quando a mesma é criada dentro de um “bloco”. Um bloco é delimitado pelos caracteres “{” e “}”. Os elementos que forem declarados dentro desse bloco farão parte desse escopo.



Funções Escopo e Passagem de Parâmetros

-> Em C#, as variáveis só podem ser acessadas no escopo em que foram declaradas. Isso significa que se declararmos uma variável dentro de uma função/método, esta variável será visível somente dentro da própria função. A tentativa de referenciar uma variável fora do seu escopo resultará em um erro de compilação.



Funções Escopo e Passagem de Parâmetros

-> Escopos podem conter outros escopos. Em outras palavras uma variável declarada em um **bloco A** que contém outro **bloco B** pode ser referenciada dentro do **bloco B**. Porém, variáveis declaradas no bloco B não podem ser referenciadas pelo bloco A.

Funções Escopo e Passagem de Parâmetros

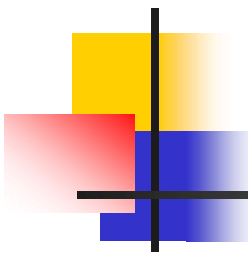
```
using System;
using System.Collections.Generic;
using System.Text;

namespace Unirondon.Celula
{
    class Program
    {
        static void Main(string[] args)
        {
            if (true)
            { // Início do escopo A
                int a = 0; // a declarada dentro do escopo A

                if (true)
                { // Início do escopo B
                    int b = 0; // b declarado dentro do escopo B
                    a = b; // Correto, a pode ser referenciada nesse escopo
                } // Fim do escopo B

                b = 1; // Errado, b não é visível ao escopo A
            } // Fim do escopo A
        }
    }
}
```

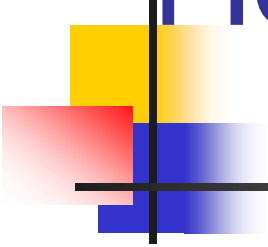
Passagem de Parâmetro por valor



```
public class Teste {  
    public static int incrementaX(int x) {  
        x = x + 10;  
        return x;  
    }  
    public static void Main(){  
        int x = 5;  
        Console.WriteLine(x); // Qual valor de x?  
        Console.WriteLine(incrementaX(x));  
        Console.WriteLine(x); // Qual valor de x?  
    }  
}
```

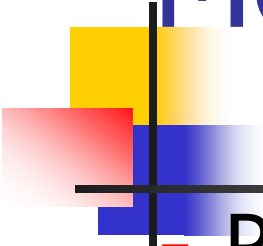
Passagem de Parâmetro por referência

```
public class Teste {  
    public static int incrementaX(ref int x) {  
        x = x + 10;  
        return x;  
    }  
    public static void Main(){  
        int x = 5;  
        Console.WriteLine(x); // Qual valor de x?  
        Console.WriteLine(incrementaX(ref x));  
        Console.WriteLine(x); // Qual valor de x?  
    }  
}
```



Modificadores

- **Modificadores:** dão informações ao compilador sobre a natureza do código, dado ou classe
 - de acesso: public, protected e private
 - Outros:
 - static



Modificadores de acesso

- Public, protected e private
- Controlam quais membros da classe podem ser acessados(). Os membros são:
 - os atributos da classe
 - os métodos e construtores da classe

Obs: somente atributos da classe podem ter modificadores de acesso. Não faz sentido ter modificadores de acesso em variáveis locais (de método) já que **o ciclo de vida** da variável está atrelada ao registro de ativação (RA) do método método.

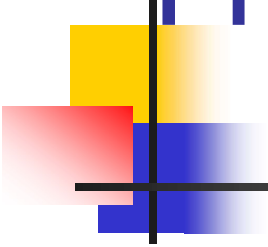


Public

- Mais liberal, todas as classes tem acesso.
A classe top-level tem que ser declarada pública.

Private

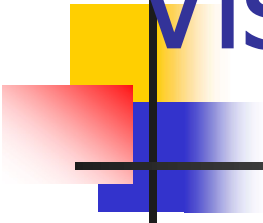
- Mais restritivo, não é acessível a ninguém.
- Atributos com private só são acessíveis dentro da própria classe



Protected

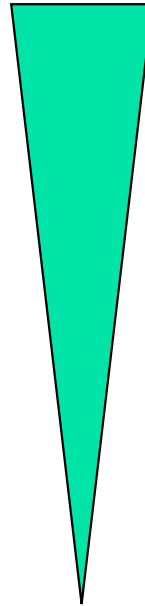
- Todas as classes do mesmo namespace tem acesso;
- Todas as subclasses tem acesso, mesmo estando em outro namespace;

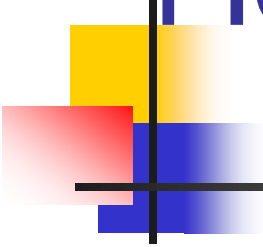
Visibilidade



◆ Quatro tipos:

- Público
- Protegido
- Pacote
- Privado

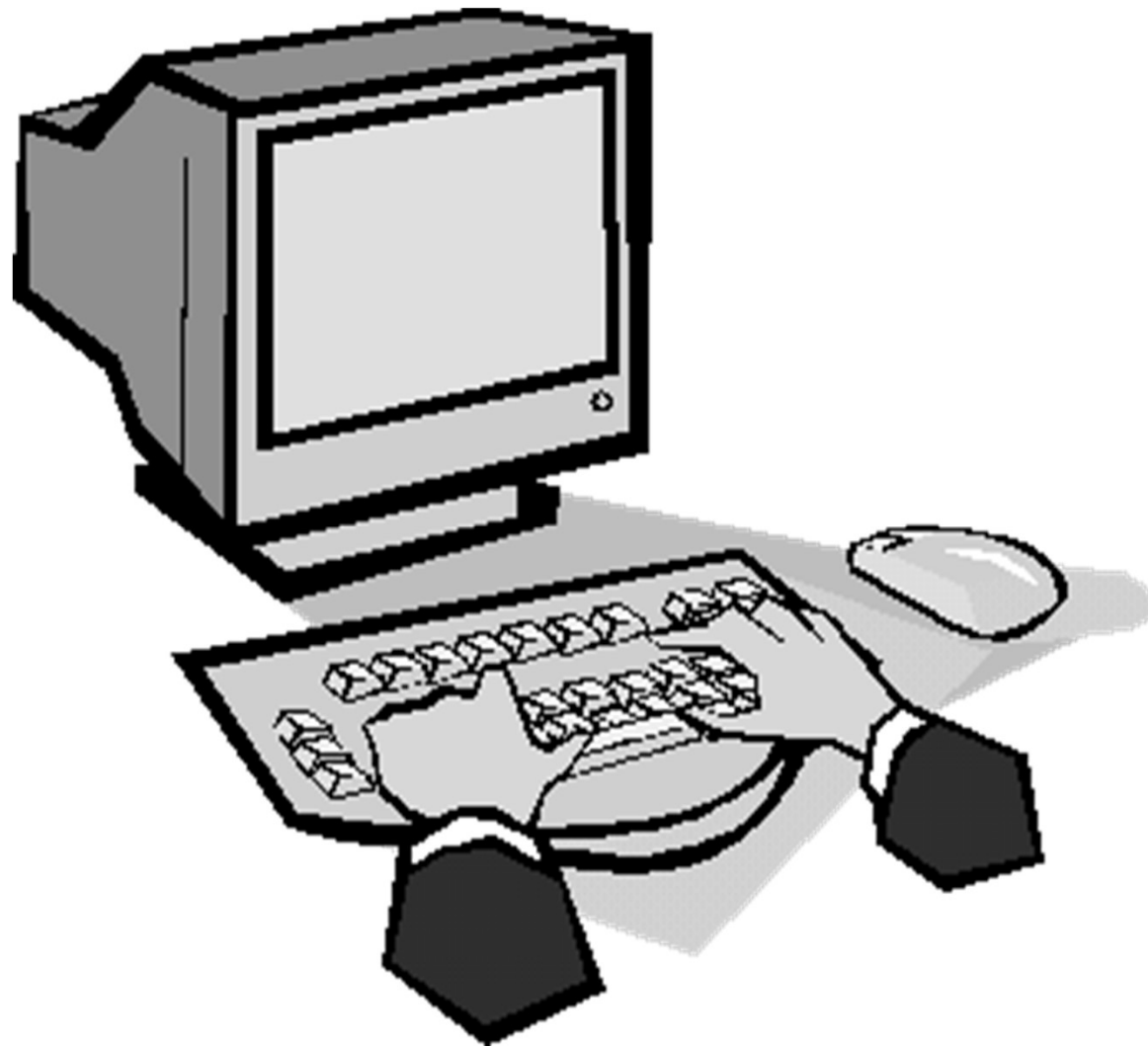




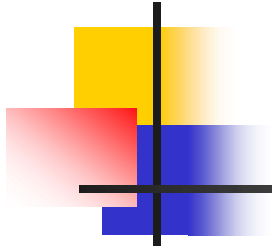
Modificadores (continuação)

- **static:** aplicado a atributos e métodos
 - o membro não está associado a uma instância da classe e sim a própria classe.
 - Não é necessário ter uma instância da classe

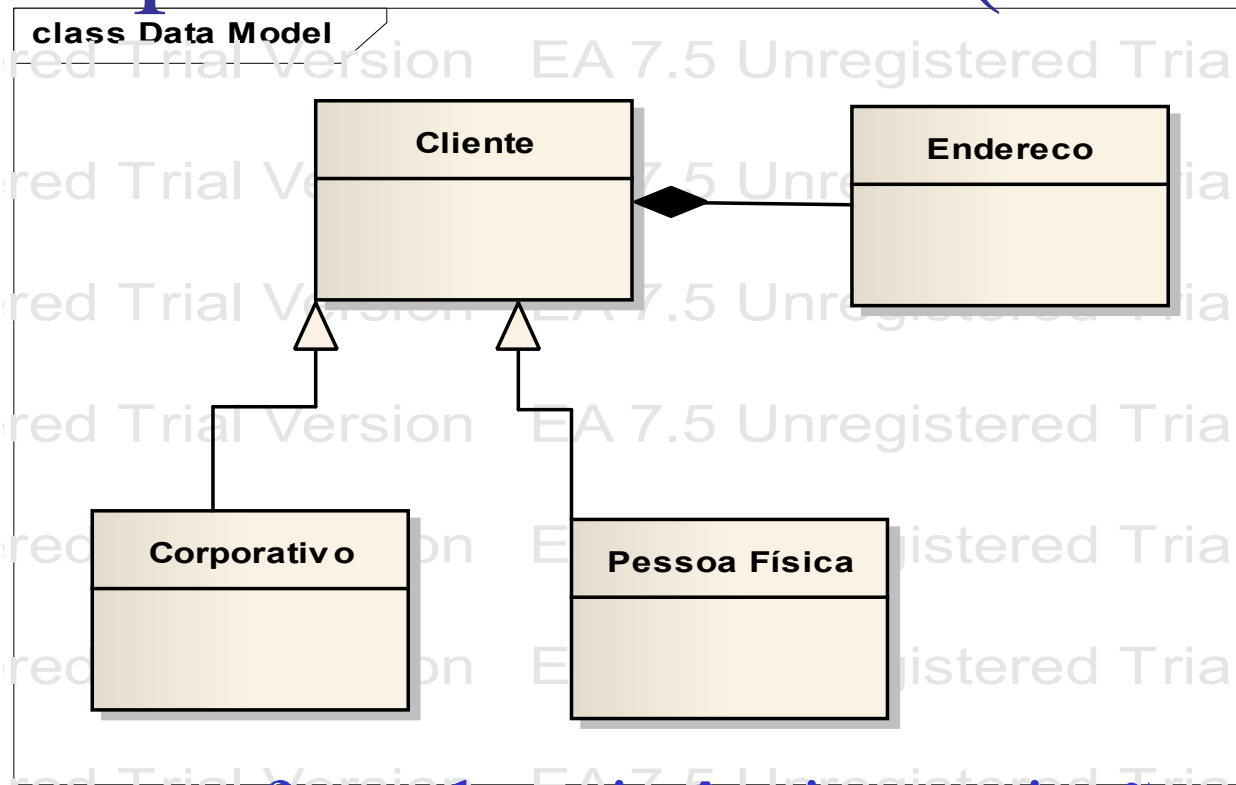
Revisão e Laboratório



Composição e associação de objetos

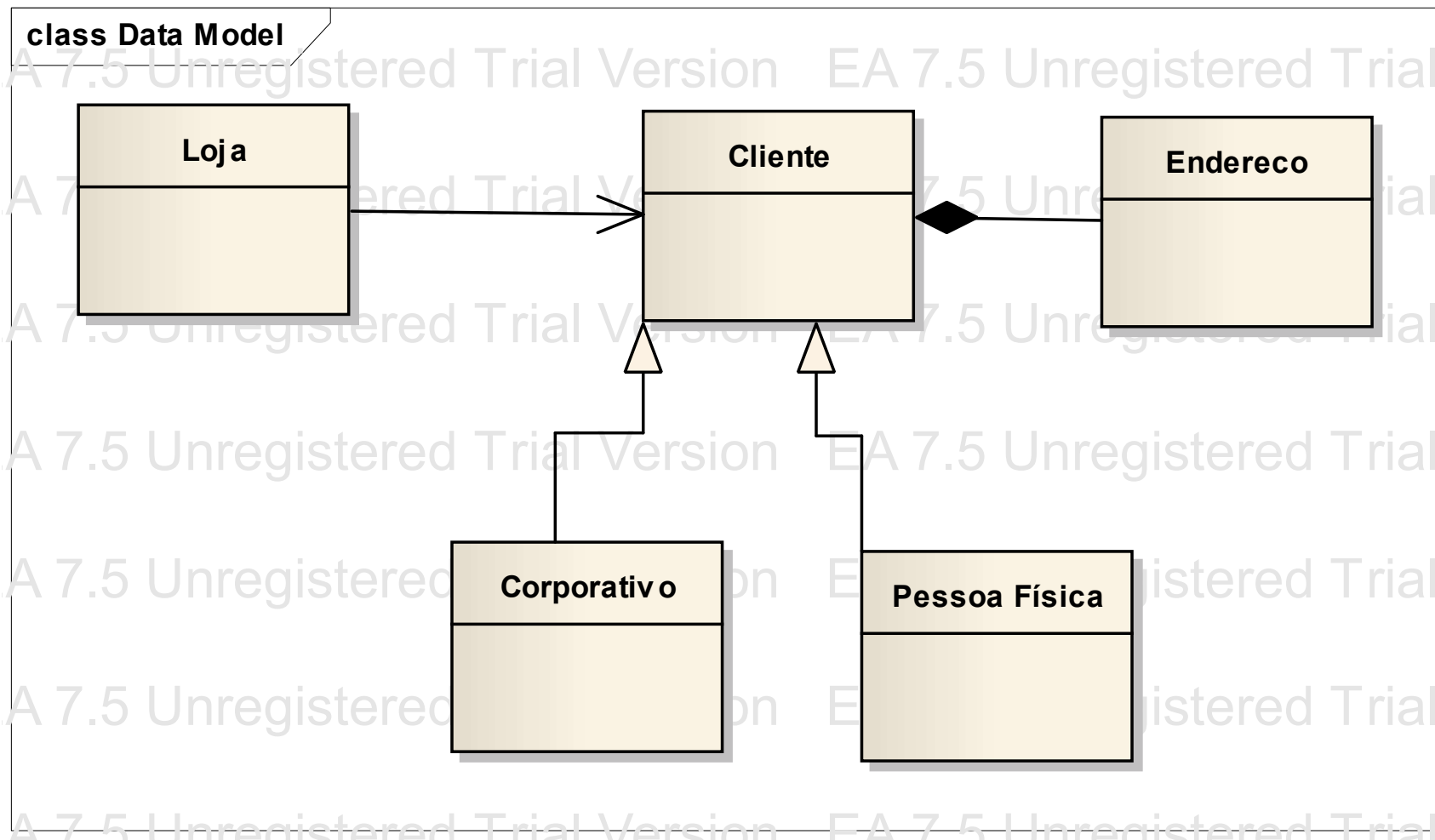


Composição: Um objeto é composto de outros (“has a”)

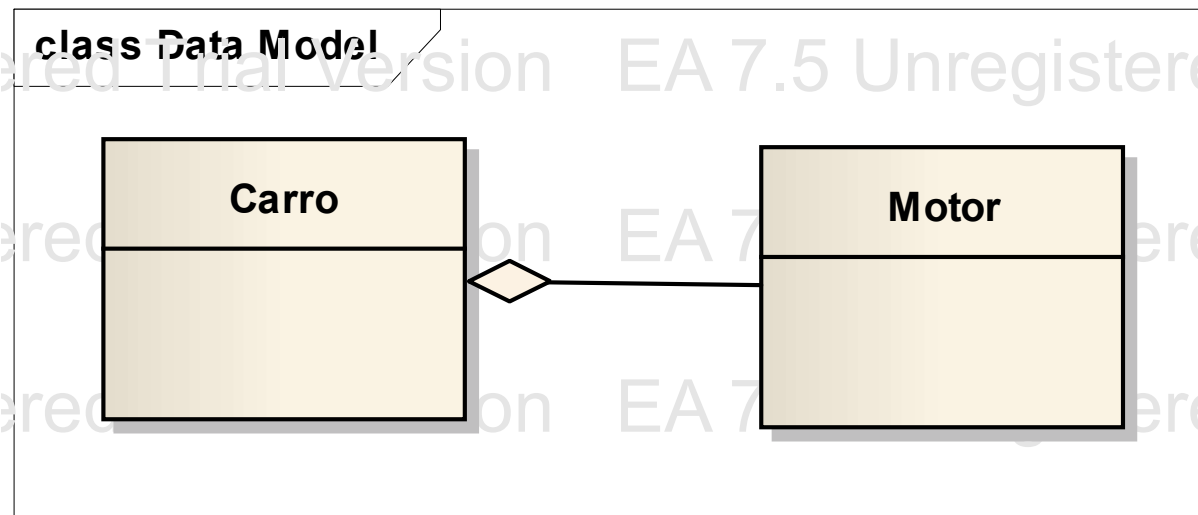


- Relacionamento forte de existência: a criação de um **cliente** implica na criação de pelo menos um **endereço**;
- Se um objeto do tipo **Cliente** for excluído, o endereço também será excluído.

Associação: Um objeto está associado a outros

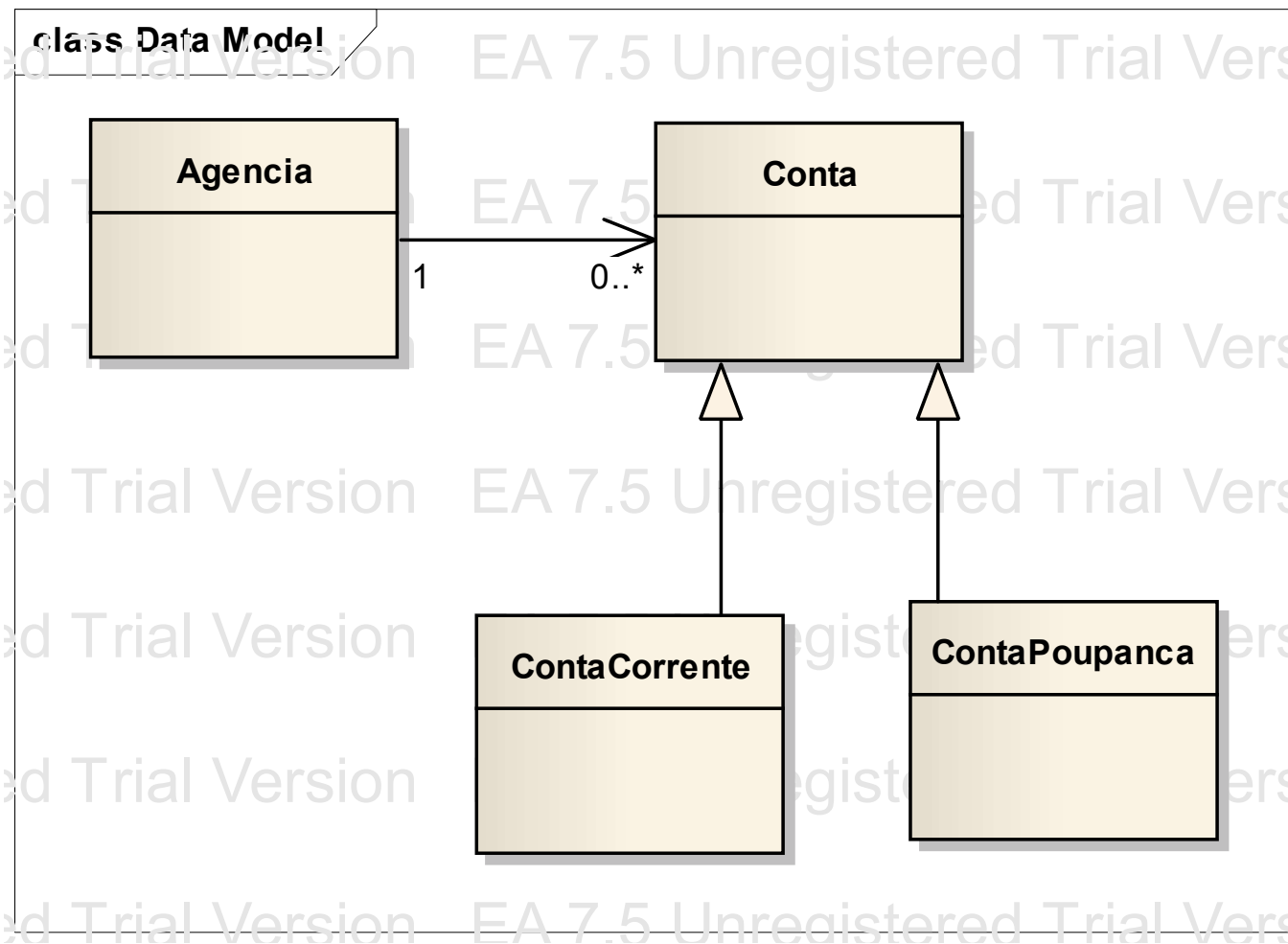


Agregação: Um objeto está associado a outros

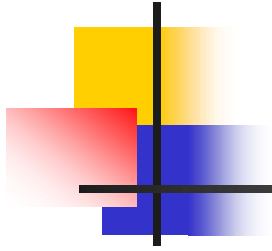


- Relacionamento fraco de existência: o ciclo de vida do **motor** não está atrelado ao ciclo de vida do **carro**, mas na maioria das vezes um carro precisa de um motor;
- A exclusão de um objeto do tipo **Carro** não necessariamente implica na exclusão do objeto do tipo Motor associado.

Laboratório: definição da classe Agencia

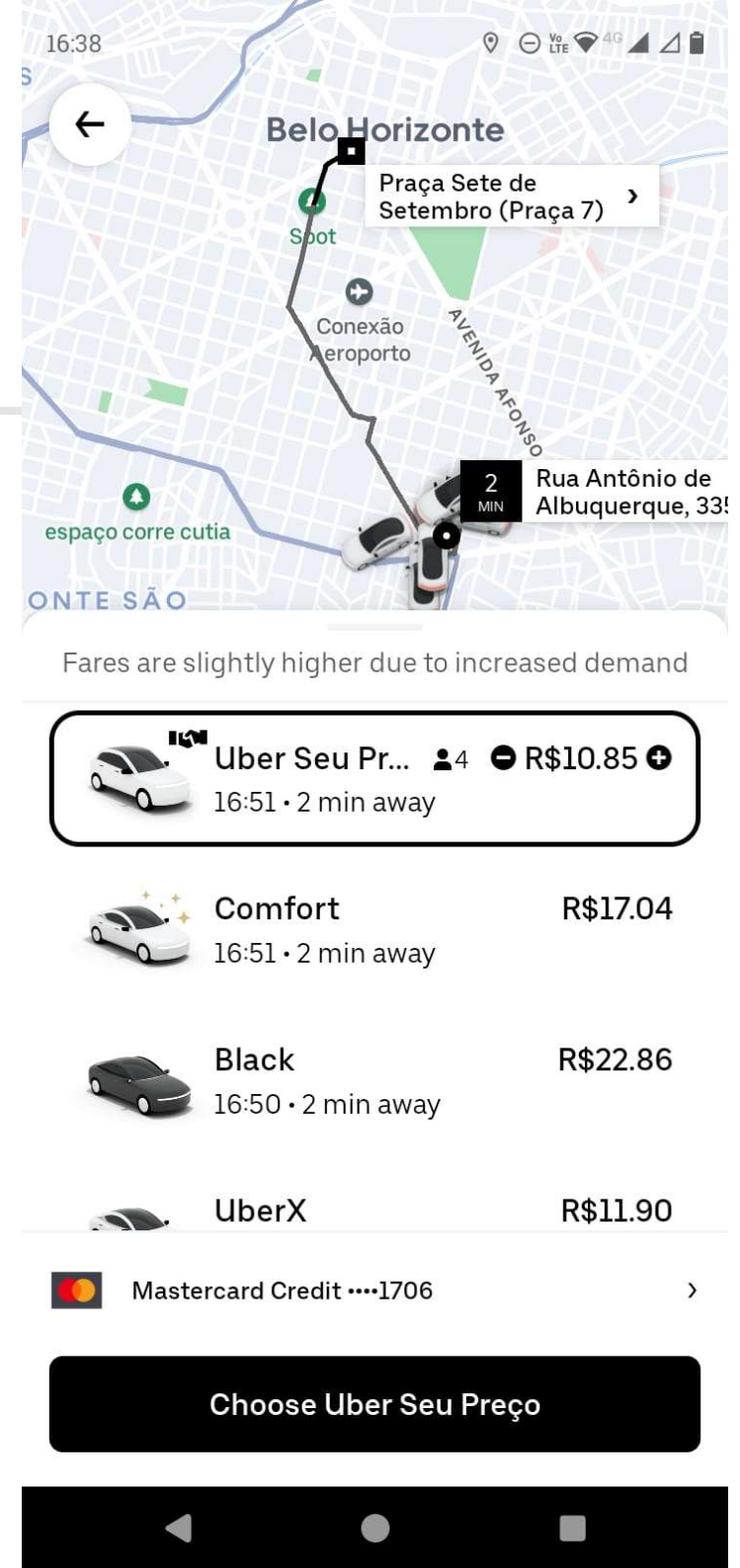


Abstração

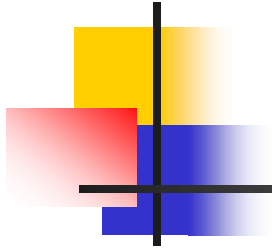


Quais objetos você
Visualiza?

- 1) carro;
- 2) motorista;
- 3) cliente;
- 4) corrida;



Abstração funcional?



Como são implementados
os serviços/operações?

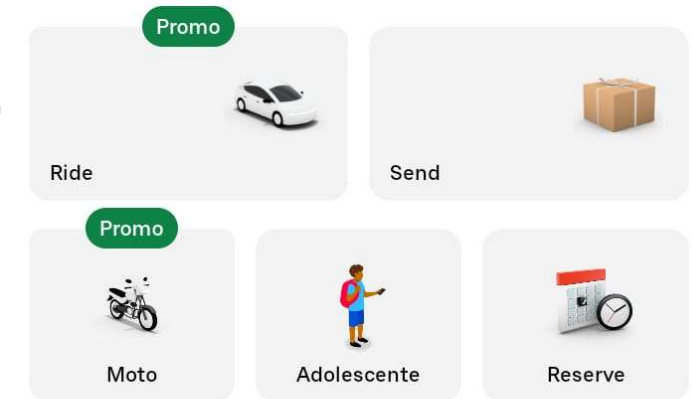
Não importa. Como cliente você vai
usar, pagar, avaliar e se for bom,
usar/contratar novamente.

16:44

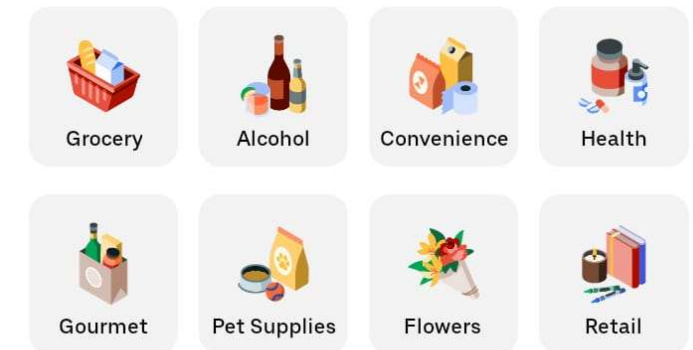


Services

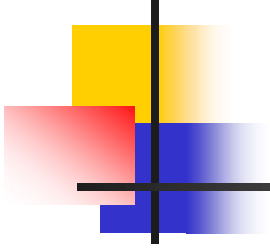
Go anywhere



Get anything delivered



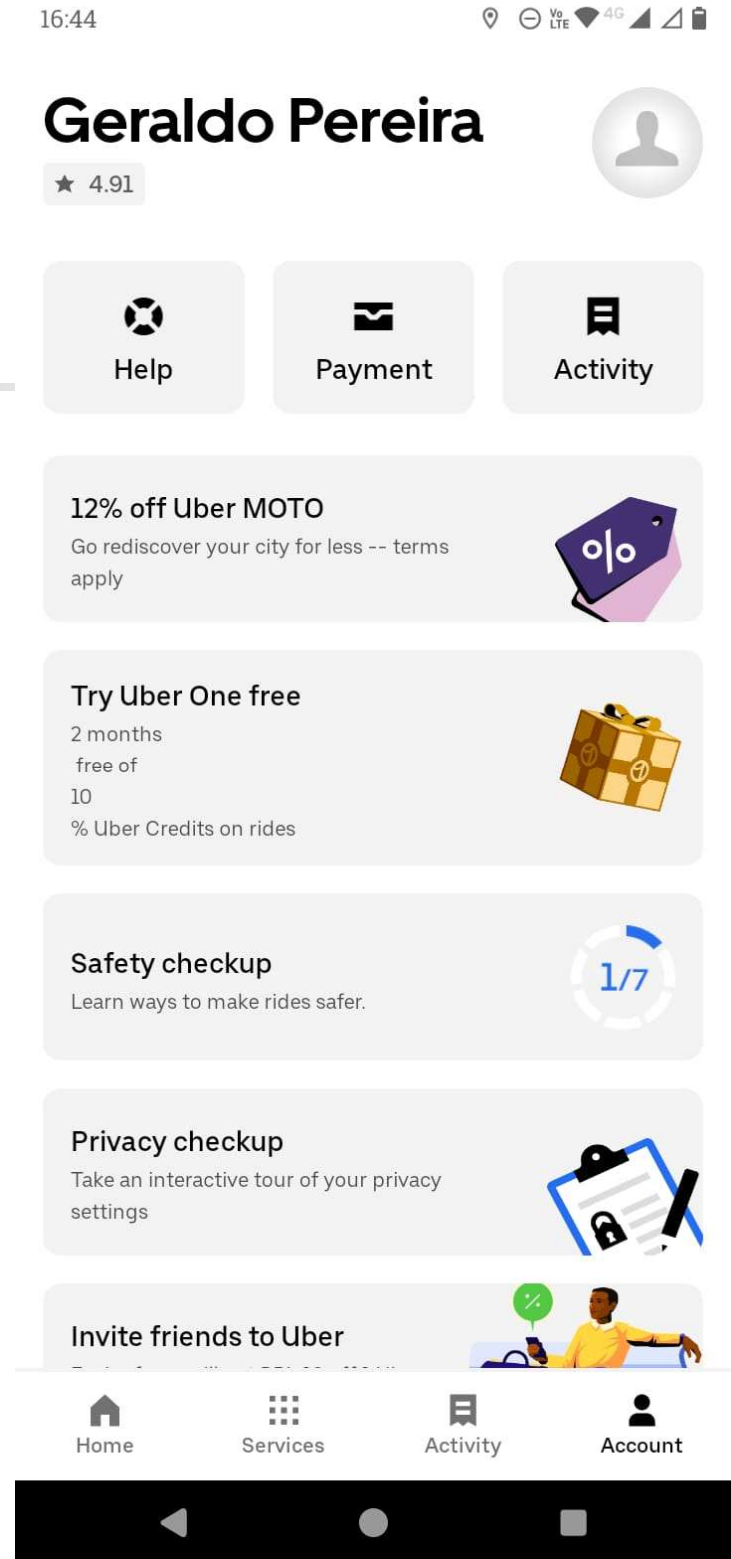
Abstração de dados



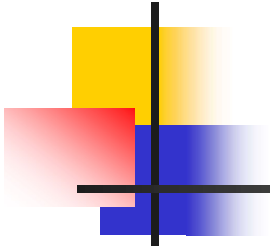
Como os dados são armazenados?
Não importa. Como cliente você vai
fornecer: os dados pessoais,
cartão, foto, etc.

Mas como esses dados são
armazenados internamente pelo
sistemas não importa para você.

Obs: LGPD.



Histórico de objetos



Por que seria importante saber o histórico de corridas?

- 1) Recuperar um item perdido; 😊
- 2) Lembrar por onde você andou; ☹️
- 3) Saber quanto você gastou; etc.
- 4) Só isso?!?!?
- 5) Use a imaginação. Encante e ofereça uma nova experiência ao usuário.

16:43



Activity

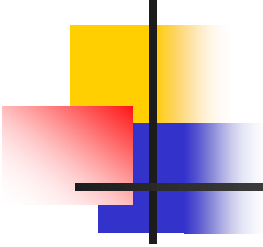
	Avenida Celso Porfírio Machado, 1520 - Belved... Aug 17 • 7:59 PM R\$42.95	 Rebook
	Avenida Celso Porfírio Machado, 1520 - Belved... Aug 17 • 7:51 PM R\$4.73 • Canceled	 Rebook
	Rua Desembargador José Burnier, 348 - Castelo Aug 17 • 3:58 PM R\$33.96 • 2 drivers	 Rebook
	Praça Sete de Setembro (Praça 7) Aug 17 • 10:15 AM R\$12.94 • 2 drivers	 Rebook
	Rua Heliotrópio, 156 - Santo André Aug 15 • 11:28 PM R\$26.05 • 2 drivers	 Rebook
	Avenida Nossa Senhora do Carmo, 600 - São Pe... Aug 5 • 1:50 AM R\$12.15	 Rebook
	R. Antônio de Albuquerque	 Rebook

Home

Services

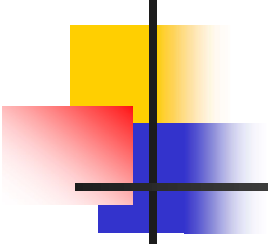
Activity

Account



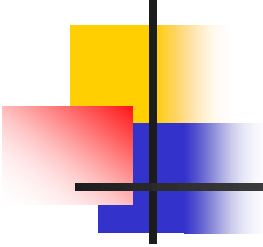
Quais os tipos das relações de objetos?

- 1) Extrato bancário e cabeçalho do extrato;
- 2) Extrato bancário e lançamentos do extrato;
- 3) Carrinho de compra e itens do carrinho de compra;
- 4) Nota fiscal e dados do comprador;
- 5) Turma e alunos;
- 6) Turma e professor;
- 7) Laboratório e PCs;
- 8) Cliente e endereço do cliente;
- 9) Conjunto e pontos do conjunto;
- 10) Outros???



Exemplo de lançamentos em arquivo posicional

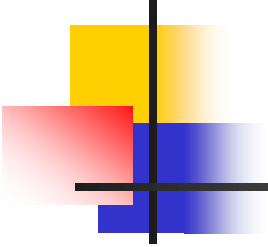
Conta;Agencia;Transacao;Data;Valor
2;300;2;09/05/2020;113.4
0;200;0;13/12/2020;424.02
4;200;0;22/03/2020;800.34
8;100;1;18/10/2024;574.05
2;200;0;04/04/2022;233.96
6;100;0;03/04/2020;238.66



Exemplo de lançamentos em arquivo posicional

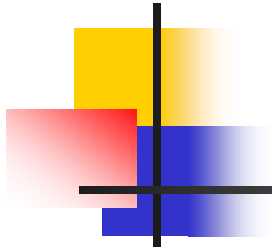
Conta;Agencia;Transacao;Data;Valor
3;300;1;1674345600;807.97
4;100;2;1680912000;872.18
3;300;0;1602460800;517.95
0;300;0;1611532800;813.01
7;300;0;1677542400;70.82
0;300;2;1712793600;256.1
2;300;2;1710979200;325.39

Exemplo de lançamentos em arquivo posicional



```
numeroConta;codigoAgencia;dataTransacao;codigoTransacao;valor;  
5;100;1645142400;3;988.1  
8;200;1589760000;0;478.5  
11;100;1598745600;2;535.1  
9;300;1606953600;0;812.3  
11;200;1636329600;2;697.3
```

Exemplo de lançamentos em arquivo posicional



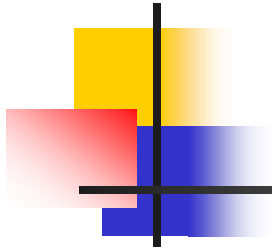
DAO (Data Access Object): classe que vai encapsular o acesso aos dados. Neste exemplo os dados estão armazenados em arquivo:

LancamentoDAO;

Controller: classe que vai encapsular as regras de negócio. Neste exemplo o cálculo do saldo final, ou o saldo de um período, serão encapsulados na classe **LancamentoController;**

View: classe que encapsular a interface com o usuário. Neste exemplo a classe Program vai fazer esse papel, em vez da classe **LancamentoView.**

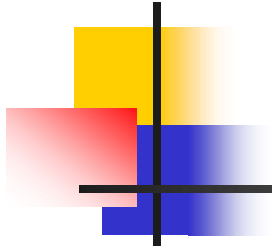
Exemplo de lançamentos em arquivo posicional



Métodos da classe **LancamentoDAO**:

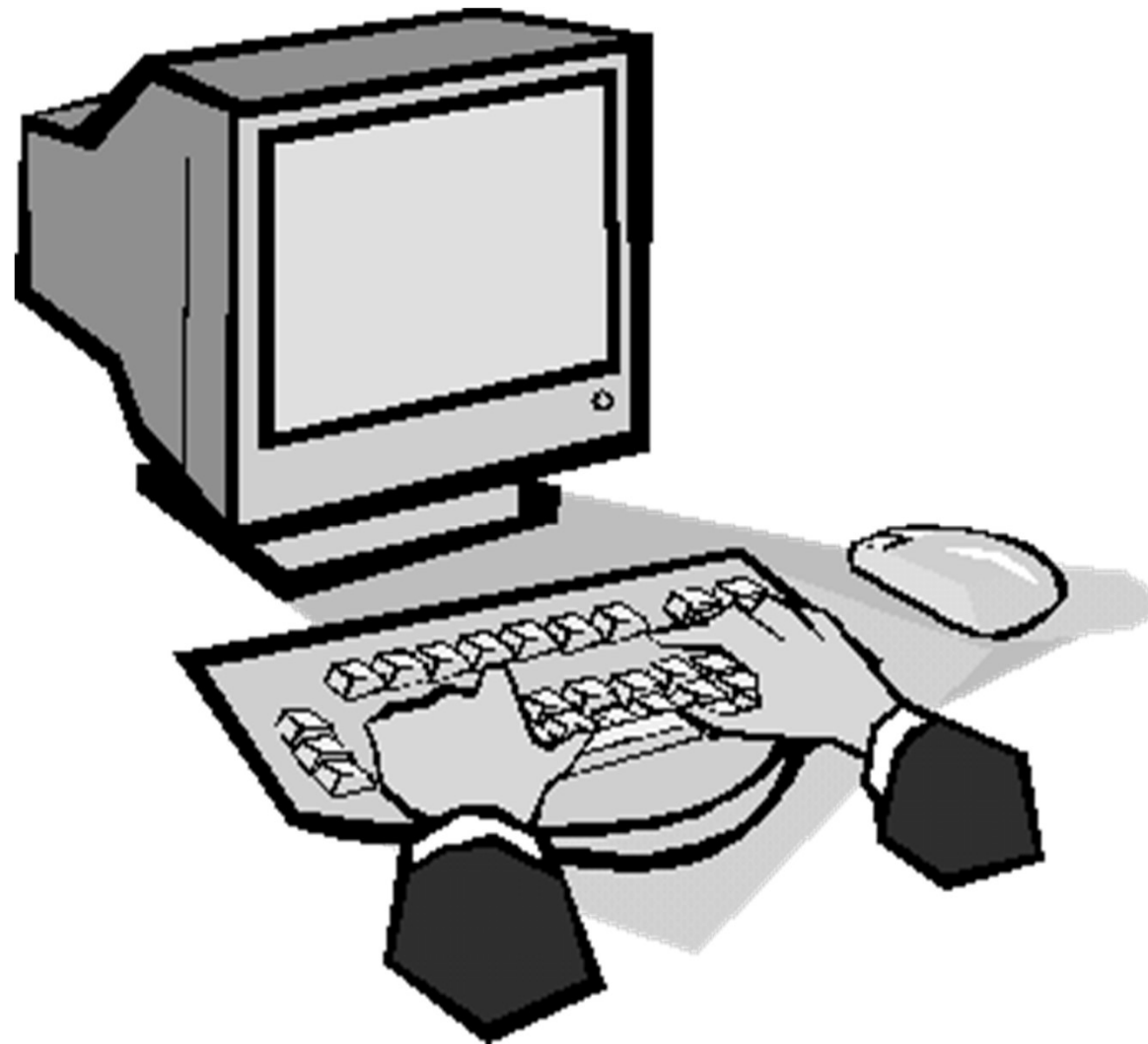
```
public void ArrayList<LancamentoVO> getTodosLancamentos();  
public void ArrayList<LancamentoVO> getLancamentos(Date  
dataInicio, Date dataFinal);  
public void ArrayList<LancamentoVO> getLancamentos(int  
numeroConta);  
public void ArrayList<LancamentoVO> getLancamentos(int  
codigoAgencia);
```

Práticas



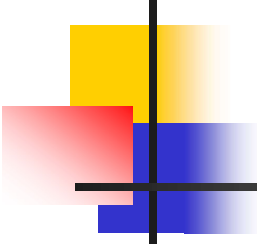
- 1) Primeiro vamos trabalhar com um exemplo mais simples de associação de objetos: prática projeto conjunto;
- 2) Posteriormente vamos implementar as classes: LancamentoVO, LancamentoDAO, LancamentoController, LancamentoView e integrá-las com a camada de visão: Program.cs.

Revisão e Laboratório



Herança de objetos





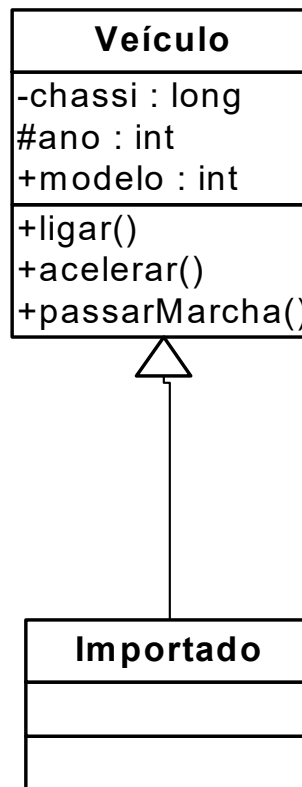
Herança

- ⇒ Herança é um mecanismo que permite a uma classe (ou seria um objeto?) herdar todo o comportamento e os atributos de outra classe (superclasse).
- ⇒ Através da herança uma classe possui imediatamente toda funcionalidade de uma classe já existente.
- ⇒ *Obs: C# não possui herança múltipla*

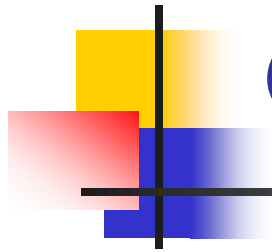
Representação na UML ("is a")

Especialização

Generalização



- ⇒ *Veículo importado é um Veículo ("is a")*
- ⇒ *Veículo tem chassi, ano e modelo ("has a")*



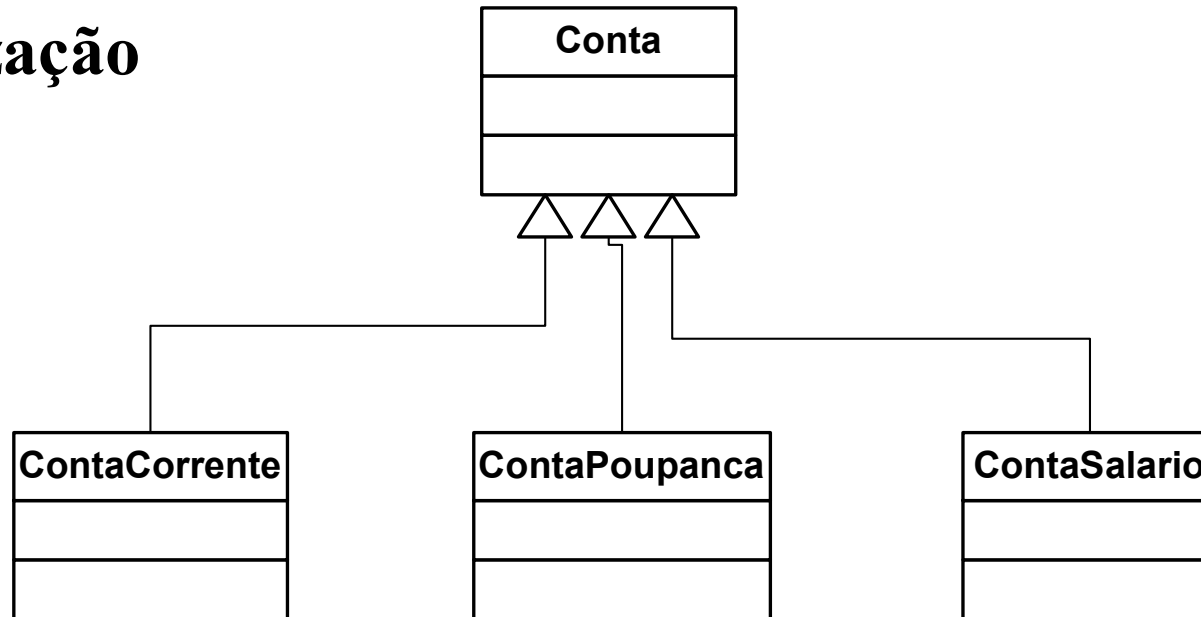
Generalização e Especialização

- **Generalização:** Atributos e funcionalidades comuns são agrupados na classe base
- **Especialização:** acrescenta-se atributos e funcionalidades específicas para uma dada classe

Diagrama de Classes

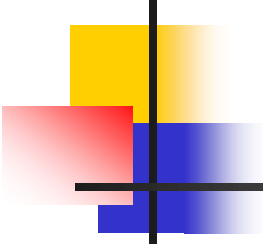
Especialização

Generalização



Conta é chamada de superclasse.

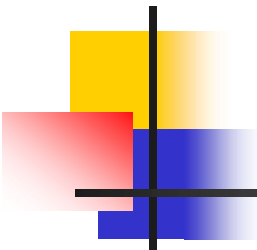
ContaCorrente, ContaPoupança e ContaSalario são subclasses de Conta.



Observações importantes

- Todas as classes no C# descendem da classe `System.Object`.
- Para dizer que uma classe é filha de outra classe utiliza-se a seguinte sintaxe:

public class ContaCorrente: Conta

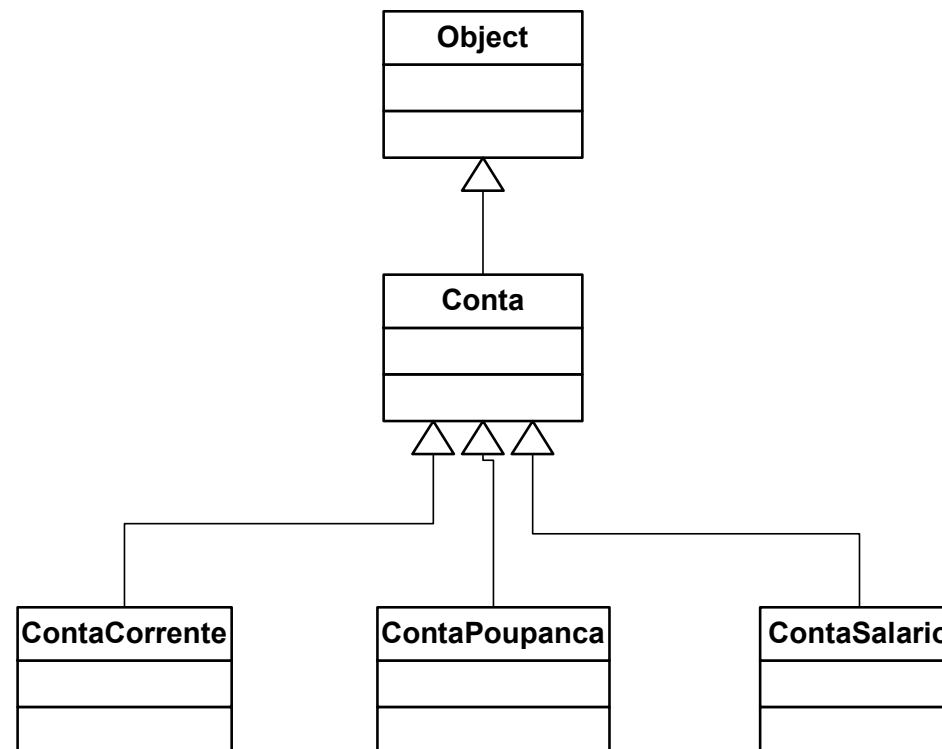


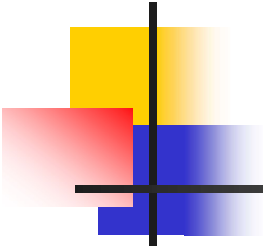
Terminologia

- **Classe** = um tipo de dados
- **estender** = criar uma nova classe que herda todo o conteúdo da classe existente (*extends*)
- **Superclasse** = classe base de outra classe
- **Subclasse** = uma classe filha que herda, ou estende, uma superclasse

Hierarquia de Classes

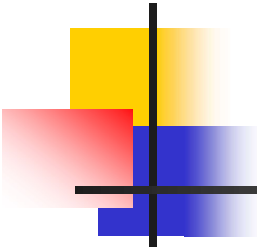
Classe "object" é raiz de todas as classes





Estudo de Caso

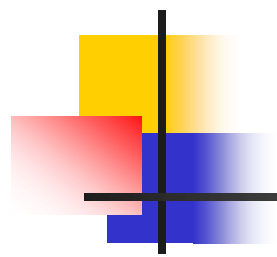
- Objetivo: Agrupar as classes do estudo de caso usando os conceitos de herança vistos até o momento. Tente elaborar o diagrama de classes do roteiro “**identificando objetos**” distribuído pelo professor.



Namespace

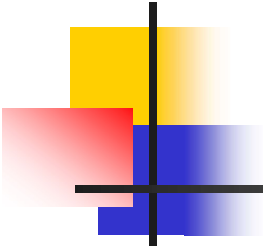
- “Namespace” é um recurso da linguagem que permite formar grupos de classes relacionadas entre si.

Localização: no início do fonte
using System;



Representação na UML





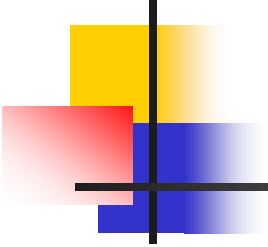
Using

Faz a inclusão de uma determinada classe de outro namespace para utilização no fonte corrente.

Localização: Após a declaração do namespace e antes da declaração da classe

Uma classe pode conter vários using

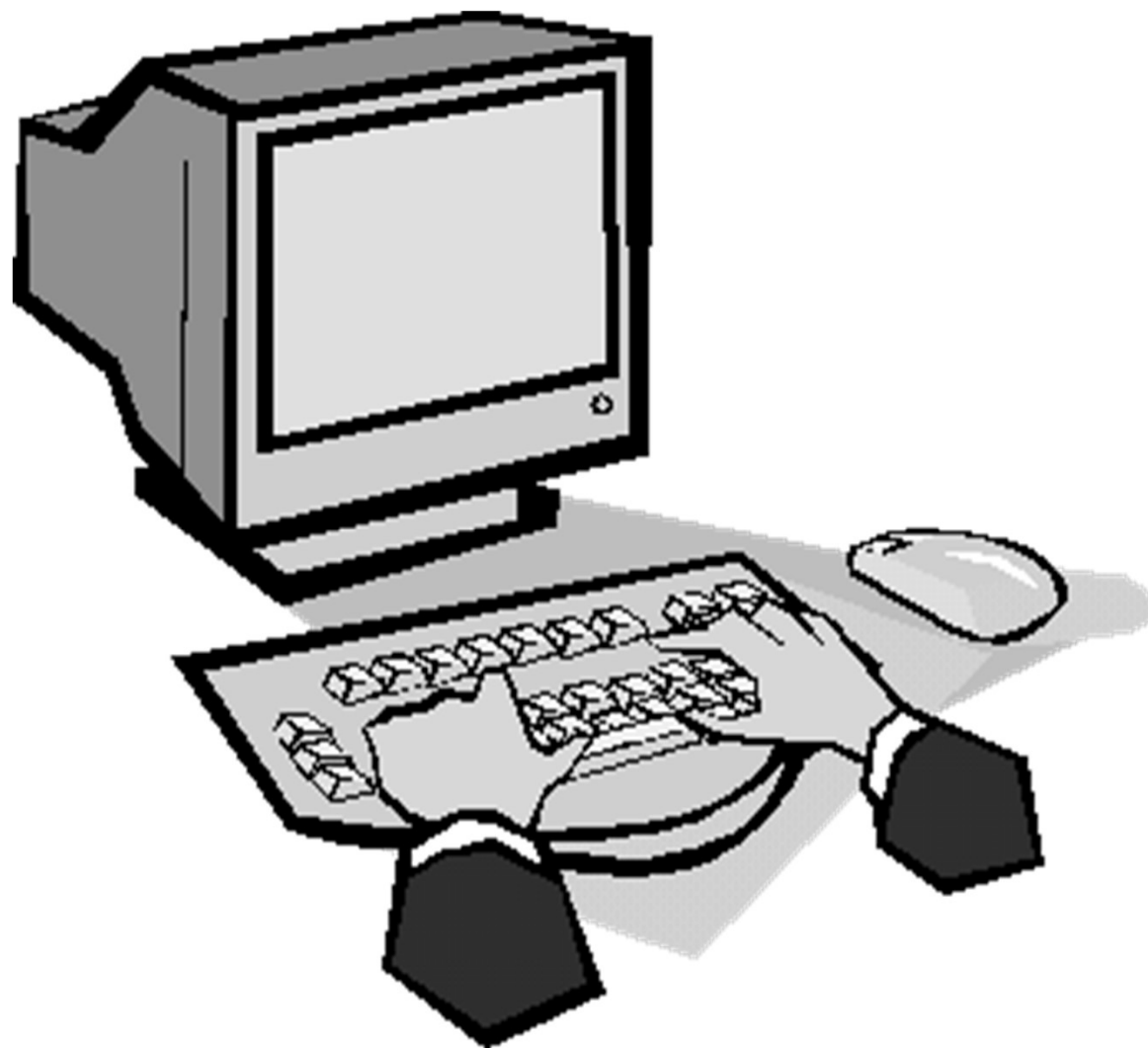
using System;



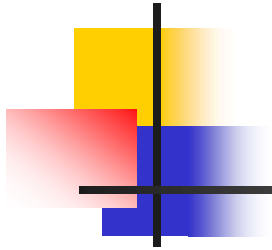
Estudo de Caso

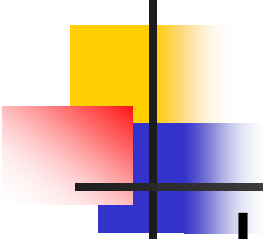
- Objetivo: Agrupar as classes do estudo de caso usando os conceitos de herança vistos até o momento. Tente elaborar o diagrama de classes do roteiro “**identificando objetos**” distribuído pelo professor.

Revisão e Laboratório



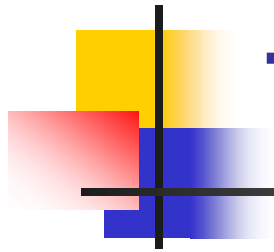
Conversão e cast de tipos básicos/primitivos





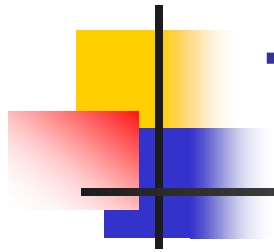
Conversão e Cast

- Uma variável tem seu tipo alterado:
 - Atribuição ou passagem de parâmetro
 - Promoção aritmética



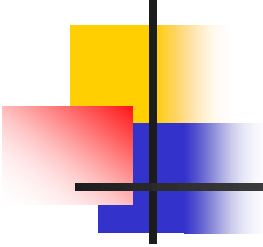
Tipo de uma variável java

- Tipo primitivo
- Referência a objetos
 - Classes
 - interface (será visto futuramente)
 - arrays de primitivos
 - arrays de objetos
 - arrays de arrays



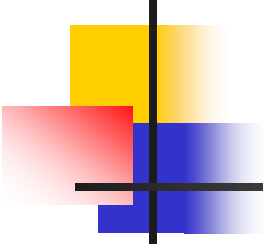
Tipos de conversão

- Implícita: feita pelo sistema
- Explícita: feita pelo programador



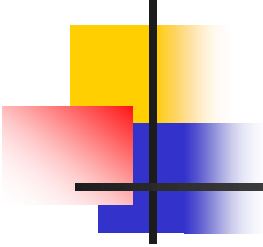
Cast e conversão (4 casos)

- Conversão de primitivos
- Cast de primitivos
- Conversão de referência a objetos
- Cast de referência a objetos



Conversão de primitivos (tempo de compilação)

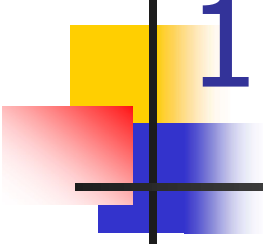
- Realizada em atribuições
- Chamada (passagem de parâmetro) de método
- Promoção aritmética



1 - Conversão na atribuição

- `int i = 10;`
- `double d = i; // ok`

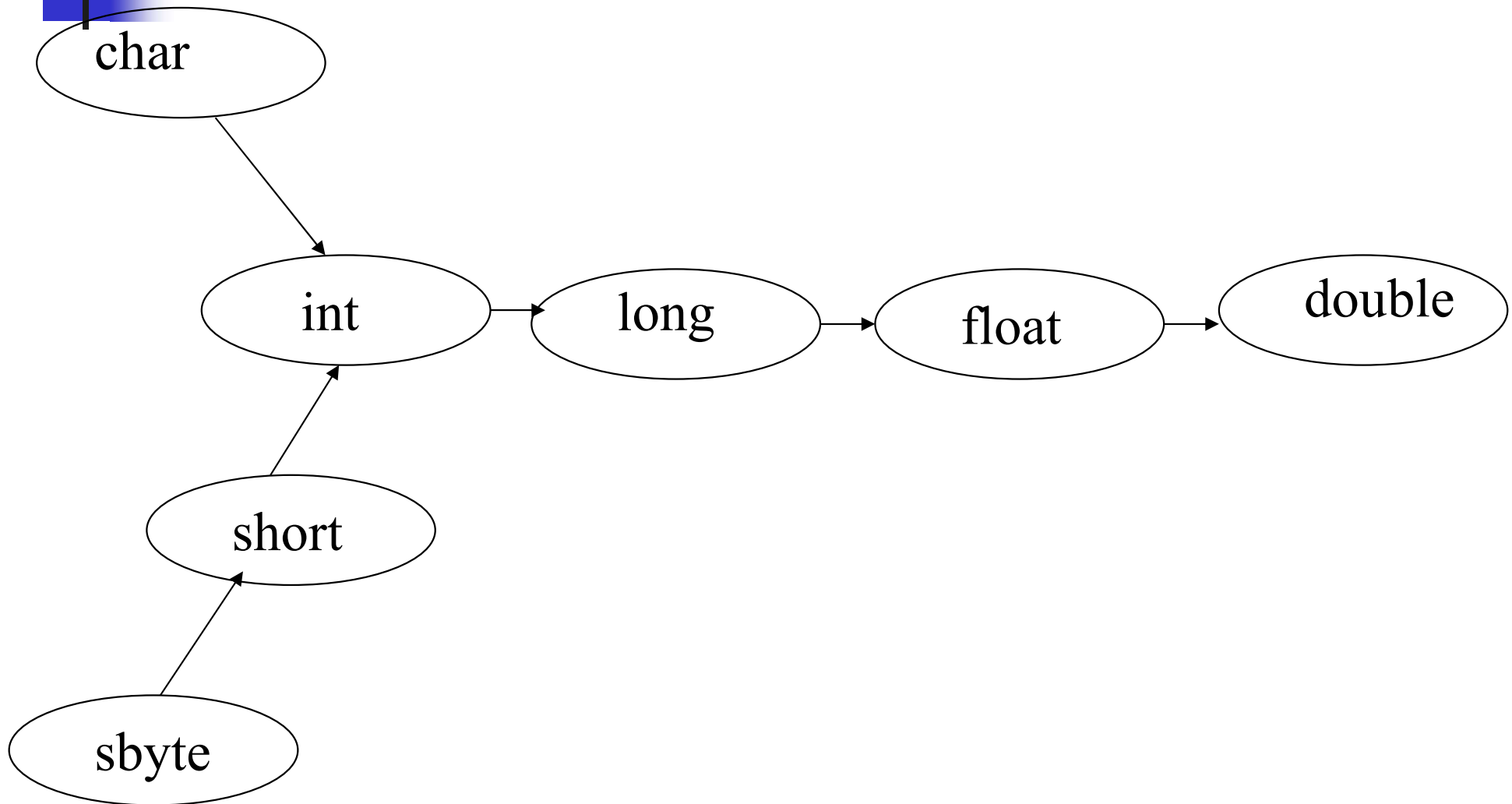
- `double d = 10;`
- `short s = d; // erro`
- Será necessário um cast explícito

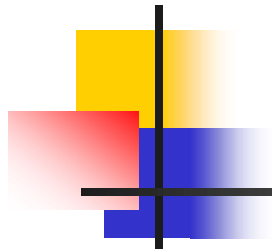


1 - Conversão de primitivos

- boolean não pode ser convertido em qualquer outro tipo primitivo
- Não boolean pode ser convertido para qualquer outro não boolean, desde que a conversão seja uma widening conversion (tipo menor para maior)
- Não boolean não pode ser convertido para qualquer outro não boolean, se a conversão for uma narrowing conversion (tipo maior para menor)

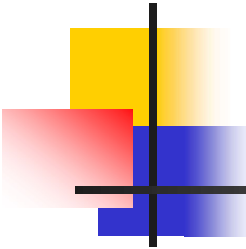
Ilustração de conversões





2 - Chamada de método

- Quando é passado um parâmetro, a conversão é realizada como na atribuição



Cast

- Tem que ser feito explicitamente pelo programador

```
int x = 10;
```

```
short y = (short) x;
```

Polimorfismo e ligação dinâmica



Polimorfismo/Ligação

Dinâmica

- ◆ Objeto polimórfico pode armazenar objetos de diferentes tipos

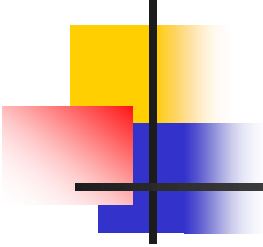
```
Conta[] listaContas = new Conta[5];  
listaContas[0] = new ContaCorrente();  
listaContas[1] = new Poupanca();
```

- ◆ Quando operações aplicadas ao tipo polimórfico, a operação associada ao tipo realmente armazenado é executada.

Polimorfismo/Ligação

Dinâmica

```
Conta c=null;
for(int i=0; i<listaContas.Length;i++){
    c=listaContas[i];
    c.imprimeDadosConta();
}
```



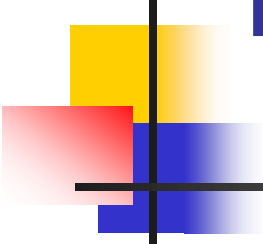
Tipos e Objetos

Construção permitida em Java porque existe herança entre as classes.

```
Conta c1 = new ContaCorrente();
```

```
ContaPoupanca p2 = new ContaPoupanca();
```

```
ContaCorrente cc3 = new ContaCorrente ();
```



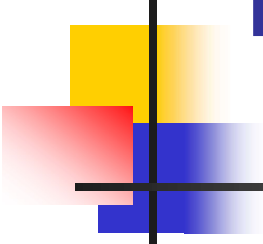
Tipos e Objetos: erros de compilação

// Erro verificado na compilação. Nem toda
conta é do tipo ContaCorrente

```
ContaCorrente cc1 = new Conta();
```

// Erro verificado na compilação. Nem toda
conta é do tipo ContaPoupanca

```
ContaPoupanca p2 = new Conta();
```



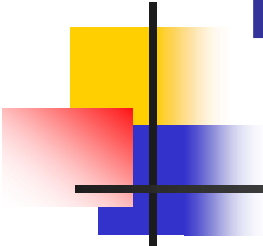
Tipos e Objetos: erros na execução

// Erro verificado na execução.

```
Conta c1 = new ContaCorrente();
```

```
ContaPoupanca p1 = (ContaPoupanca) c1;
```

Erro só verificado durante a execução no momento do CAST.



Tipos e Objetos:

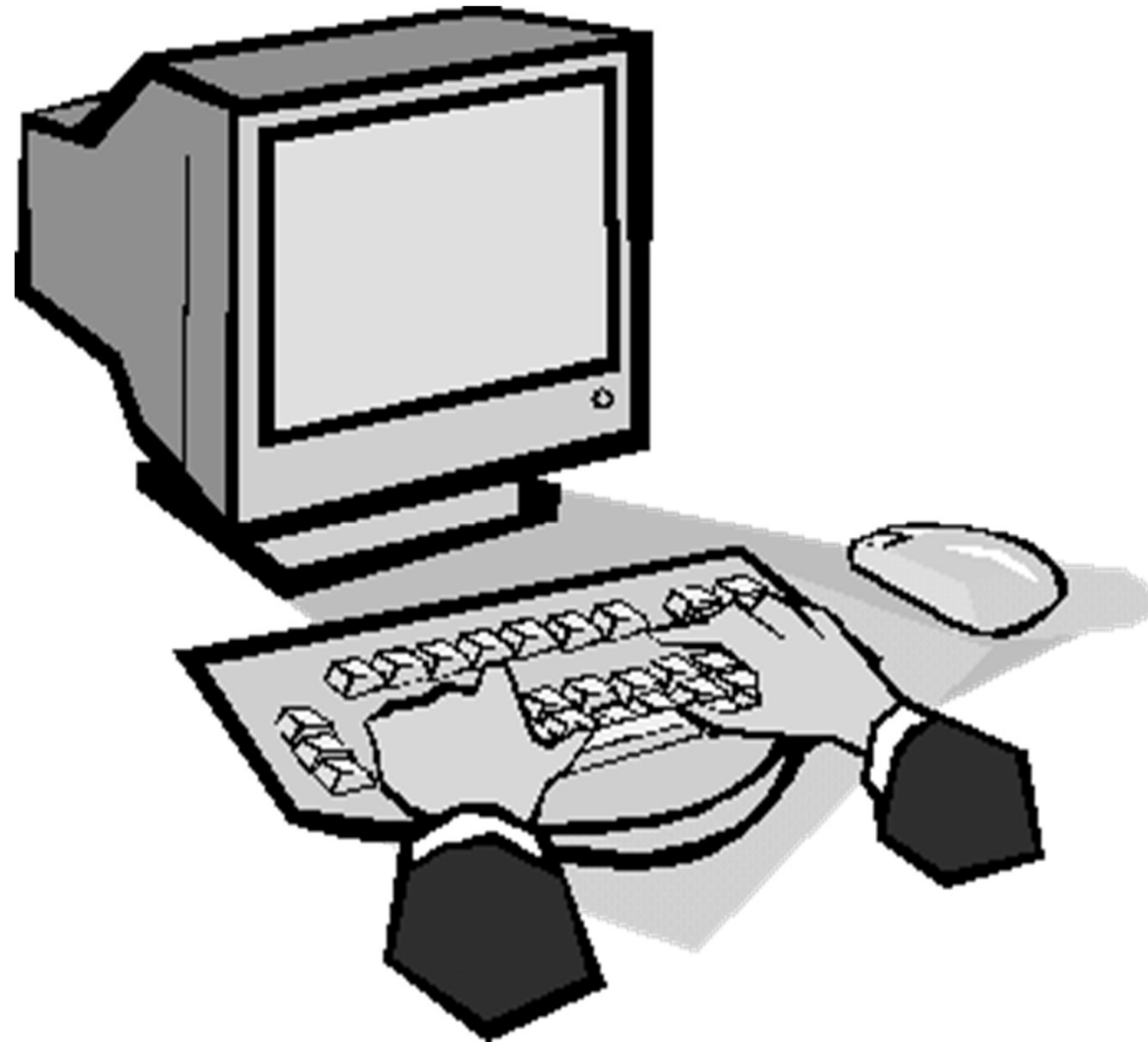
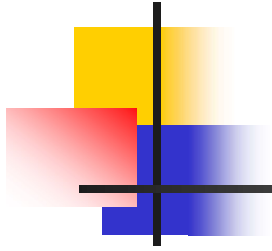
// Erro verificado na execução.

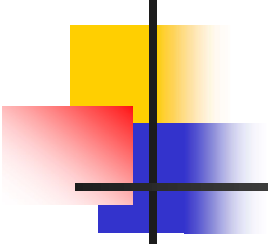
```
Conta c1 = new ContaCorrente();
```

```
ContaCorrente cc1 = (ContaCorrente ) c1;
```

Não ocorrerá erro na execução porque o objeto instanciado corresponde ao tipo de cc1 que é do tipo **ContaCorrente**.

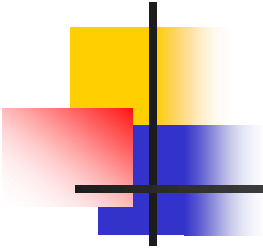
Revisão e Laboratório





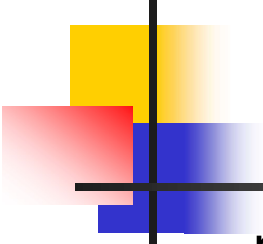
Módulo 07 - Exceções

- Tratamento de exceções

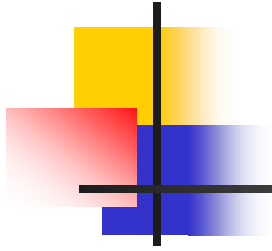


Exceções

- Condições de erro que podem ocorrer nos programas;
- O erro pode derrubar/abortar o programa ou pode ser tratado pelo programador;
- Alguns erros são difíceis de serem tratados pelo programador: falta de memória ou um erro físico no hd, por exemplo. Em **sistemas distribuídos** são apresentadas estratégias para contornar tais situações.



```
namespace ConsoleApplication1{
    class Program{
        static void Main(string[] args){
            int i = 0;
            string [] msg = {"Olá mundo","Não, eu digo!","OLÁ MUNDO"};
            while(i<4)
            {
                Console.WriteLine(msg[i]);
                i++;
            }
        }
    }
}
```



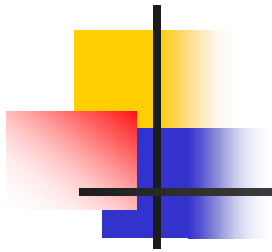
Resultado do programa:

“Olá mundo”

”Não, eu digo!”

”OLÁ MUNDO”

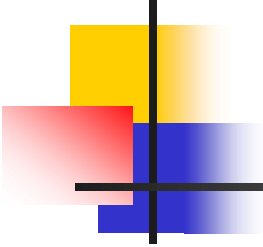
Erro na posição 4!!!



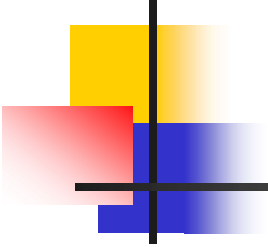
System.IndexOutOfRangeException was unhandled
Message="O índice estava fora dos limites da matriz."
Source="ConsoleApplication1"

StackTrace:

em ConsoleApplication1.Program.Main(String[] args) na
G:\PUC2\2010-
1\ATP\publicado\pratica\LabFiles_CSharp\ProjetoFuncao\Console
Application1\ConsoleApplication1\Program.cs:linha 16
em System.AppDomain._nExecuteAssembly(Assembly
assembly, String[] args)



```
static void Main(string[] args){
    int i = 0;
    string [] msg = {"Olá mundo","Não, eu digo!","OLÁ MUNDO"};
    while(i<4){
        try{
            Console.WriteLine(msg[i]);
        }
        catch (Exception e){
            Console.WriteLine("Erro no programa");
        }
        finally{
            i++;
        }
    }
    Console.ReadKey();
}
```



Tratamento

try

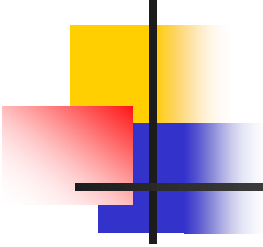
{“Código a ser executado e sujeito a erro”}

catch(<Exceção>)

{“Procedimento em caso de erro”}

finally

{“Sempre executado”}



Erros comuns

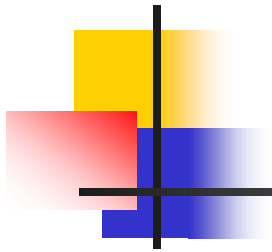
- `NullPointerException`: objeto não instanciado
- `IndexOutOfRangeException`: índice inválido no array

Erros comuns: no java

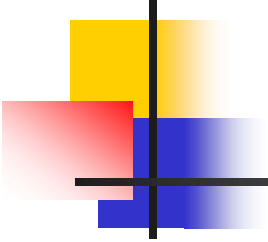


```
java.lang.NullPointerException
    at srvLogin_INTRA.service(srvLogin_INTRA.java:528)
    at com.sssw.srv.busobj.AgoServletInvoker.serviceRequest(AgoServletInvoker.java:111)
    at com.sssw.srv.resources.AgServletResource.service(AgServletResource.java:157)
    at com.sssw.srv.resources.AgServletResource.perform(AgServletResource.java:114)
    at com.sssw.srv.http.httpd.perform(httpd.java:5631)
    at com.sssw.srv.http.Client.processRequest(Client.java:902)
    at com.sssw.srv.http.Client.loop(Client.java:1268)
    at com.sssw.srv.http.Client.runConnection(Client.java:1484)
    at com.sssw.srv.http.Client.run(Client.java:1432)
    at java.lang.Thread.run(Thread.java:479)
```

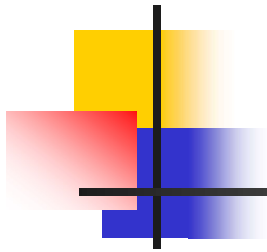




```
static void Main(string[] args)
{
    int i = 0;
    string nome = null;
    string sobreNome = nome.Substring(5, 4);
    Console.ReadKey();
}
```

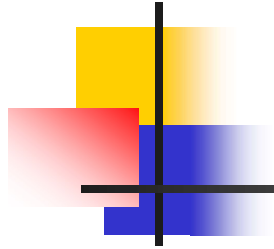
```
static void Main(string[] args){  
    int i = 0;  
    string nome = null;  
    try  
    {  
        string sobreNome = nome.Substring(5, 4);  
    }  
    catch (Exception e)  
    {  
        Console.WriteLine("Erro " + e);  
    }  
  
    Console.ReadKey();  
}
```



Erro System.NullReferenceException: Referência de objeto não definida para uma instância de um objeto.

em ConsoleApplication1.Program.Main(String[] args) na
G:\PUC2\2010-1\ATP\publicado\pratica\LabFiles_CSharp\ProjetoFuncao\ConsoleApplication1\ConsoleApplication1\Program.cs:linha 16

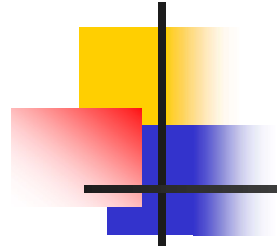
Como criar uma classe Exception?



[Serializable]

```
public class InvalidDepartmentException : Exception {  
    public InvalidDepartmentException() : base() { }  
    public InvalidDepartmentException(string message) : base(message) { }  
    public InvalidDepartmentException(string message, Exception inner) : base(message,  
inner) { }  
}
```

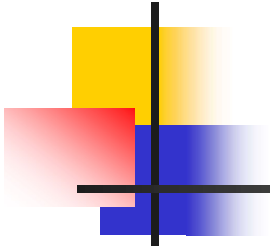
Como lançar uma Exception?



```
public class ProgramLog{
    FileStream logFile = null!;
    public void OpenLog(FileInfo fileName, FileMode mode) { }

    public void WriteLog(){
        if (!logFile.CanWrite)
        {
            throw new InvalidOperationException("Logfile cannot be read-only");
        }
        // Else write data to the log and return.
    }
}
```

Como lançar uma Exception?



```
public static Task<Toast> ToastBreadAsync(int slices, int toastTime) {  
  
    if (slices is < 1 or > 4) {  
        throw new ArgumentException( "You must specify between  
            1 and 4 slices of bread.", nameof(slices));  
    }  
  
    if (toastTime < 1) {  
        throw new ArgumentException( "Toast time is too  
            short.", nameof(toastTime));  
    }  
  
    etc;  
}
```

No java tem o conceito de Checked Exception e C# não.

```
public void debitar(double valor) throws SaldoInvalidoException
{
    System.out.println("Debito em conta");
    if (saldo > valor){
        System.out.println("Saldo anterior = " + saldo);
        saldo = (saldo - valor);
        System.out.println("Saldo atual  = " + saldo);
        System.out.println("Valor do debito= " + valor);
        System.out.println("Debito confirmado! ");
    }
    else{
        throw new SaldoInvalidoException("Saldo inválido");
    }
}
```

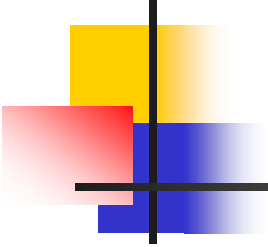
No java tem o conceito de Checked Exception e C# não.

```
public class SaldoInvalidoException extends Exception{  
    private String causa;  
    // Pode incluir outros atributos conforme projeto  
    public SaldoInvalidoException(String causa){  
        this.causa = causa;  
    }  
    public String getMessage(){  
        return causa;  
    }  
}
```

No java tem o conceito de Checked Exception e C# não.

```
public static void main(String[] args){
    Conta contaJoao = new Conta(1, 1000);
    contaJoao.saldo();
    // Linha abaixo não irá compilar pois está fora de try/catch
    contaJoao.debitar(500);
    try{
        contaJoao.debitar(500);
        contaJoao.saldo();
    }
    catch(SaldoInvalidoException e){
        System.out.println(e.getMessage());
    }
    catch(Exception e){
        System.out.println(e.getMessage());
    }
    contaJoao.saldo();
}
```

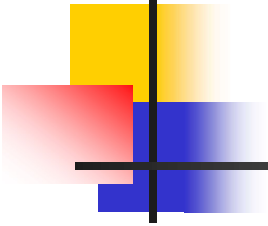

O que é "calar" uma Exception?



```
public static void main(String[] args){
    Conta contaJoao = new Conta(1, 1000);
    contaJoao.saldo();
    // Linha abaixo não irá compilar pois está fora de try/catch
    contaJoao.debitar(500);
    try{
        contaJoao.debitar(500);
        contaJoao.saldo();
    }
    catch(SaldoInvalidoException e){
        // Exceção será ignorada. Não há nenhum código aqui.
    }
    catch(Exception e){
    }
    contaJoao.saldo();
}
}
```

Mais sobre Exceções





Manipulação de Arquivos em C#

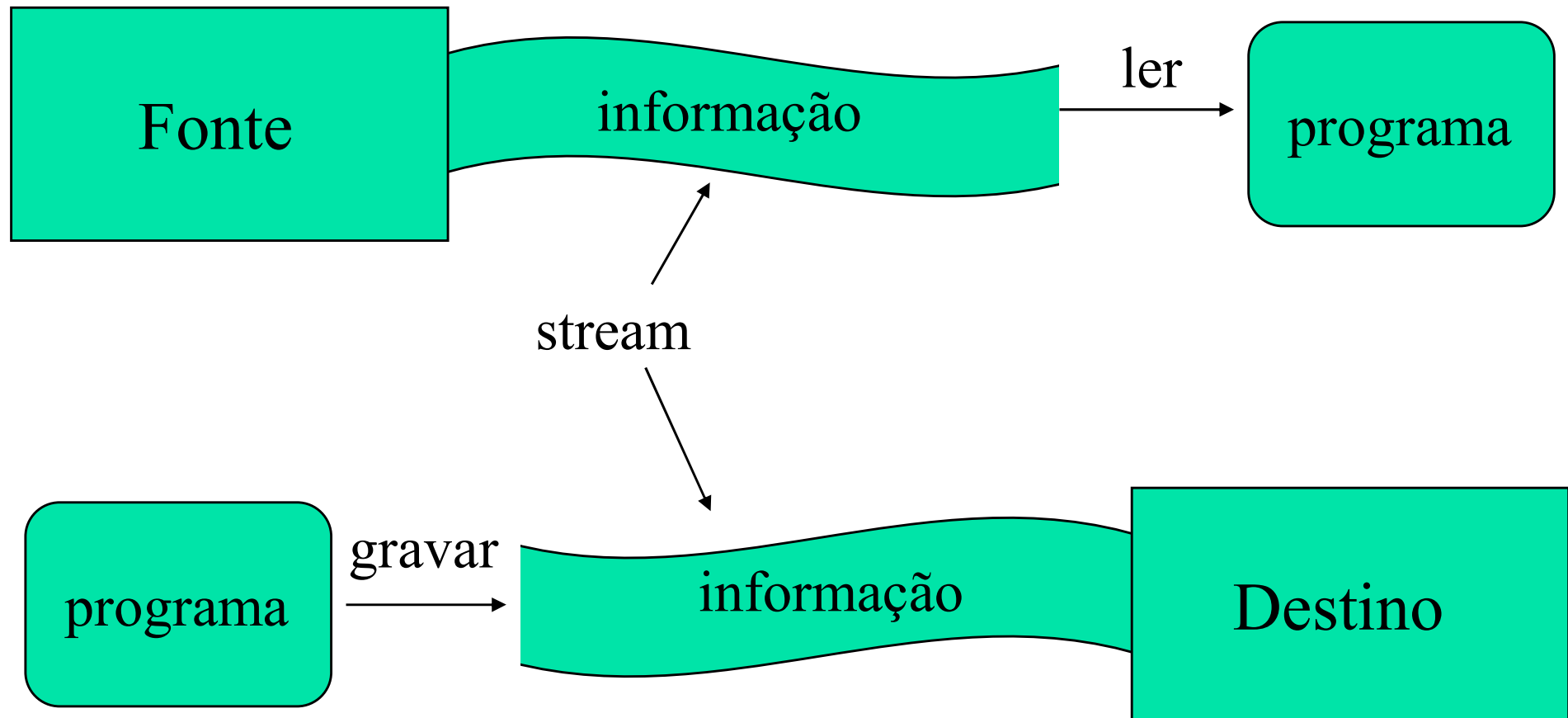
Introdução ao Sistema de IO no C#



Manipulação de informações

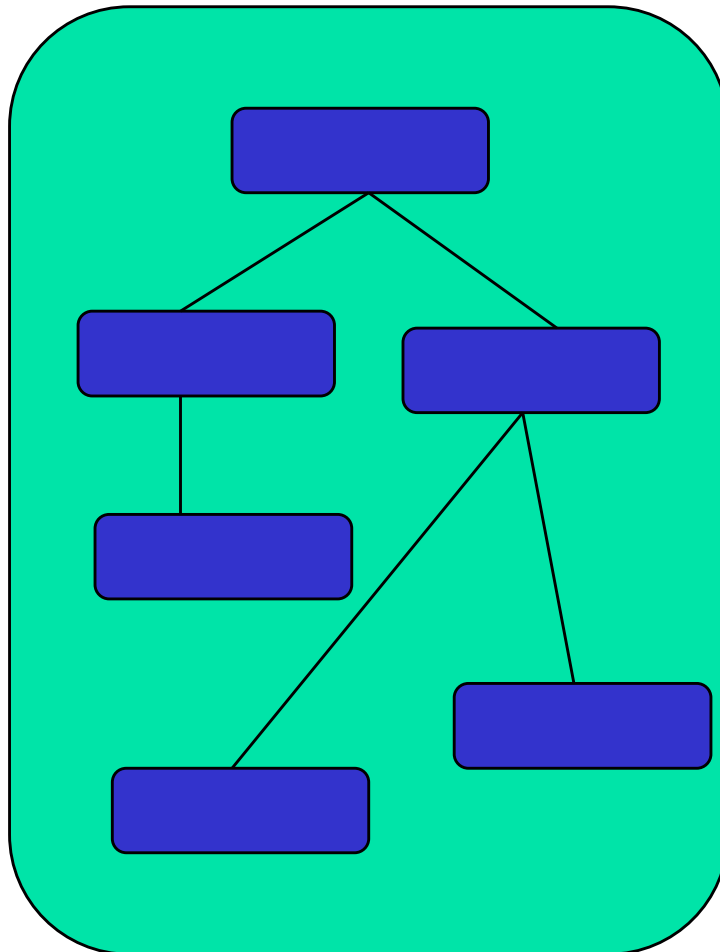
- Às vezes um programa precisa buscar e/ou enviar informações para um fonte externa;
- As informações podem estar em:
 - Em um Sistema Gerenciador de Banco de Dados (SGDB);
 - Em um arquivo;
 - em algum lugar na rede;
 - em outro programa.
- As informações podem ser dos seguintes tipos:
 - objetos;
 - caracteres;
 - Imagens;
 - Sons etc.

Um programa lendo ou gravando informações em canais de *stream*

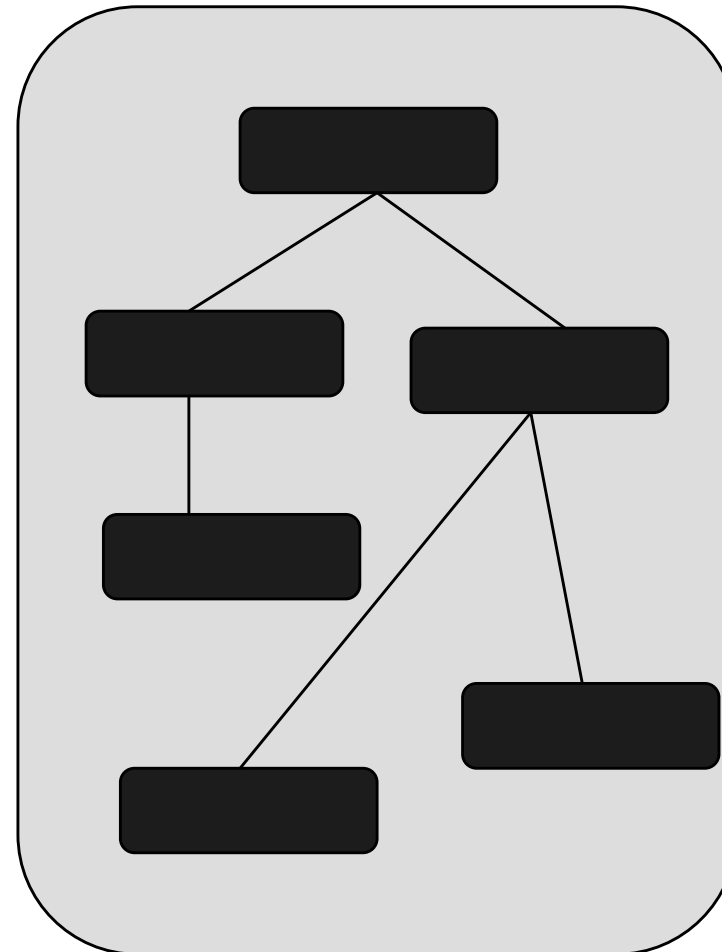


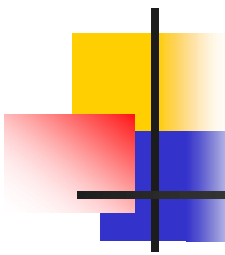
O pacote System.IO contém duas hierarquias de classes independentes

Stream de caracter



Stream de byte





Um exemplo de programa para gravar em arquivo:

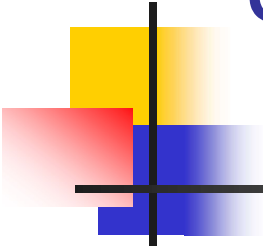
Passo 1: Importar o pacote System.IO

using System.IO;

Passo 2: Criar um objeto do tipo Writer:

*StreamWriter wr = new
StreamWriter(@"c:\pasta\arquivo.txt", true);*

Obs: Colocar o @ antes do path do arquivo faz com que o compilador não interprete a barra "\" como sendo uma mudança de linha ou tabulação (\n ou \t) e sim como uma string.



Um exemplo para gravar em arquivo:

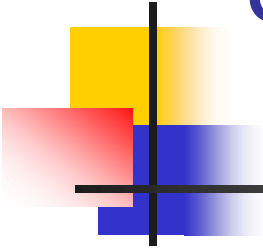
Passo 3: Escrever no arquivo:

A seguir podemos escrever no arquivo a função de escrita:

```
wr.WriteLine("Este é o texto a escrever no arquivo");
```

Passo Final: Todo recurso aberto deve ser fechado e liberado após o uso.

```
wr.Close();
```

Um exemplo para ler de arquivo:

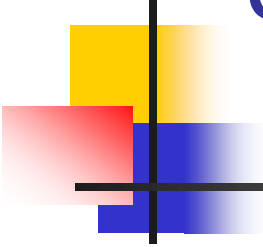
Para a leitura de arquivos, pode-se utilizar a classe *StreamReader*.

Passo 1: Abrir o arquivo:

```
StreamReader rd = new StreamReader(@"c:\pasta\ficheiro.txt");
```

Passo 2: Ler informações do arquivo:

```
while(!rd.EndOfStream){  
    string linha = rd.ReadLine();  
    Console.WriteLine(linha);  
}
```



Um exemplo para ler de arquivo:

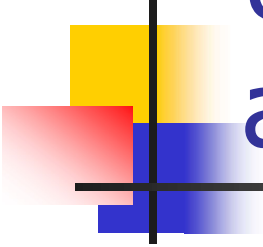
Passo Final: Todo recurso aberto deve ser fechado e liberado após o uso.

rd.Close();



Um exemplo para gravar e ler de arquivo do tipo texto:

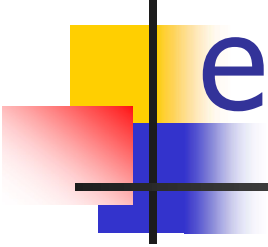
```
StreamWriter wr = new StreamWriter(@"D:\doc.txt", true);
wr.WriteLine("Primeira linha");
wr.WriteLine("Segunda linha");
wr.Close();
StreamReader rd = new StreamReader(@"D:\doc.txt");
while (!rd.EndOfStream){
    string linha = rd.ReadLine();
    Console.WriteLine(linha);
}
rd.Close();
Console.Write("\n\n\nPressione qualquer tecla para sair...");
Console.ReadKey(true);
```



Um exemplo para gravar e ler de arquivo do tipo binário:

@todo: em construção para ser trabalhado na aula prática.

Fundamentos Orientação à Objetos e Programação Utilizando C#



Obrigado!

Professor: Geraldo Pereira de Souza

Obs: Esses slides foram produzidos para implementação em linguagem java e estão sendo atualizados para C#. Caso verifique algum erro ou em caso de dúvidas e/ou sugestões favor informar ao professor.